

Yolov11原理和代码详解

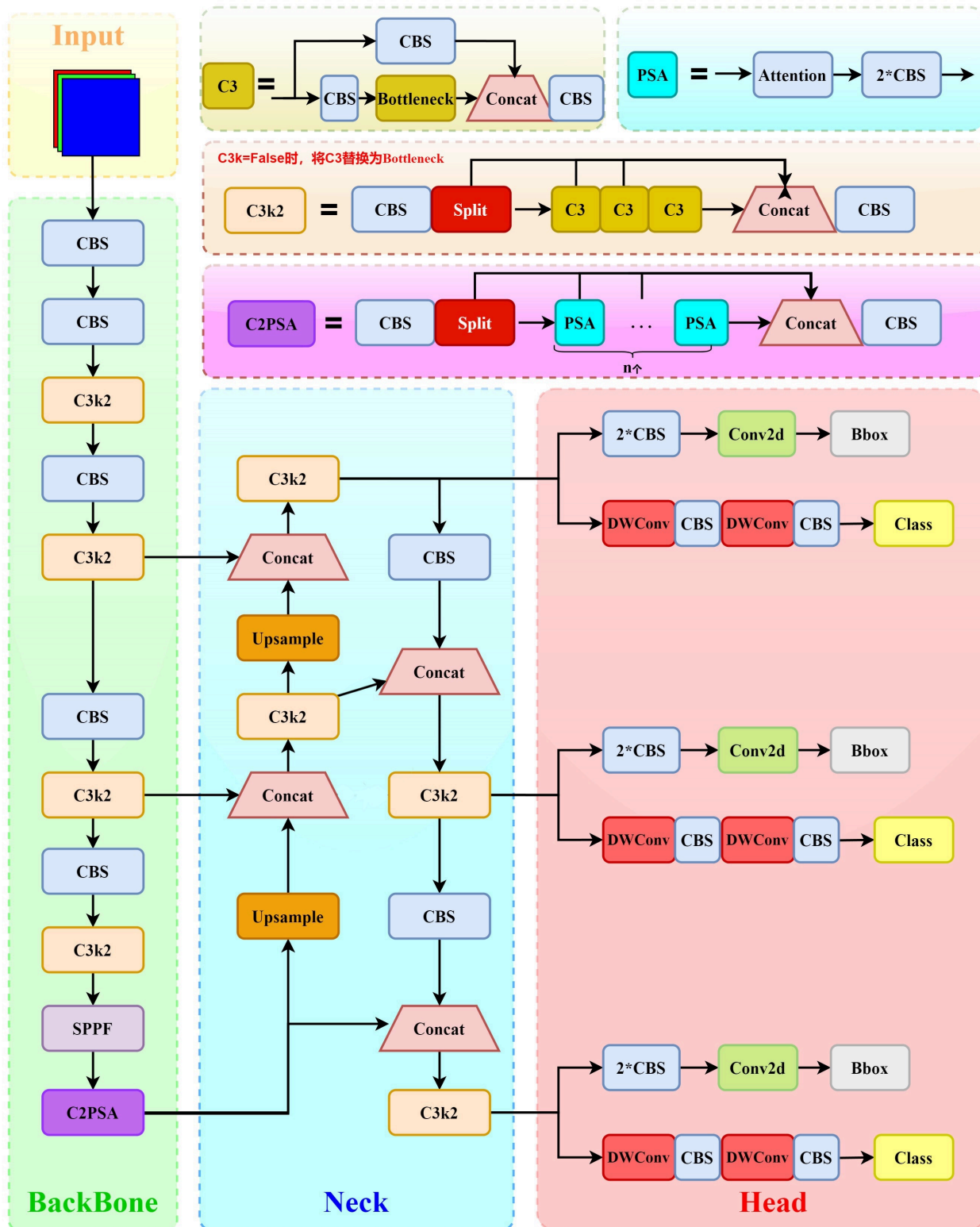
1.yoloV11架构

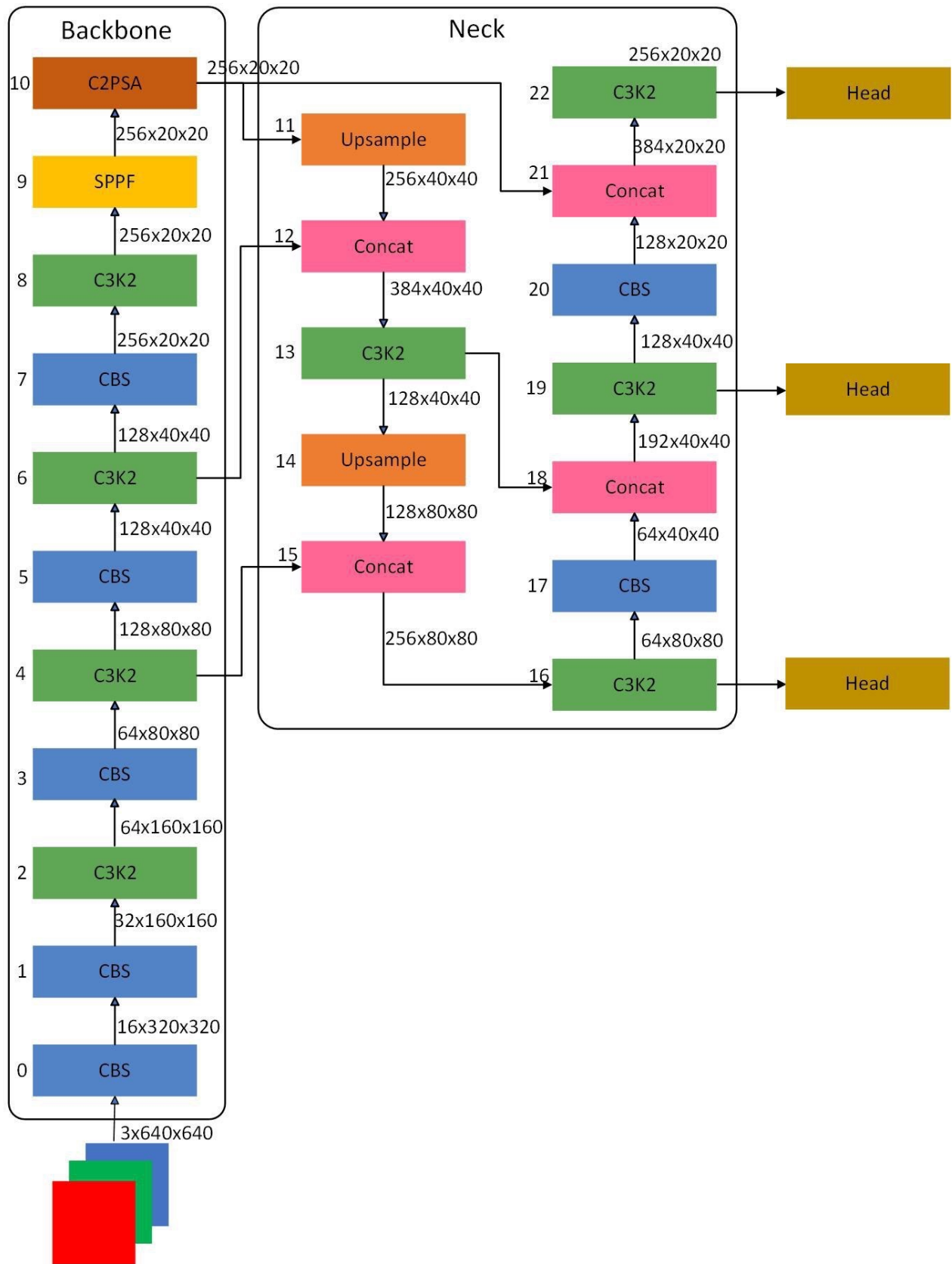
1.1 模型介绍

代码地址: <https://github.com/ultralytics/ultralytics>

YOLOv11继承自YOLOv8, 在YOLOv8基础上进行了改进, 使同等精度下参数量降低20%, 在速度和准确性方面具有无与伦比的性能。其流线型设计使其适用于各种应用, 并可轻松适应从边缘设备到云 API 等不同硬件平台。使其成为各种物体检测与跟踪、实例分割、图像分类和姿态估计任务的绝佳选择。

1.2 架构图





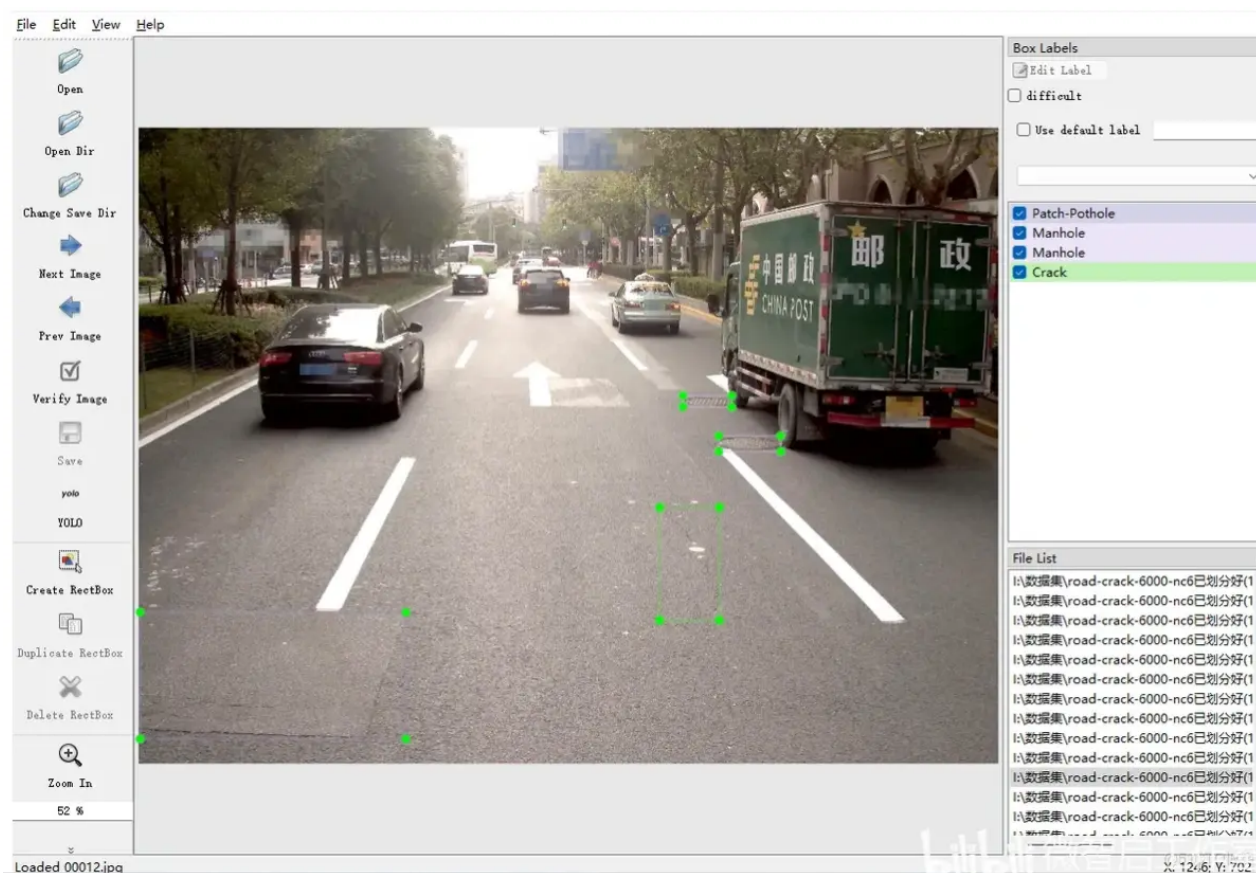
2. 项目准备

2.1 环境准备

- 首先创建虚拟环境
- 然后 `pip install ultralytics` 会自动安装所需要的包

2.2 数据介绍

路面裂缝、井盖、坑洼路面的标识数据集，使用labelimg标注，标签的格式是txt格式，适用于yolo目标检测系列所有版本训练数据集。



数据集总共有9个类，已经划分好训练集和验证集。

训练的配置参数信息如下

```
nc: 9
```

```
names: ["Crack", "Manhole", "Net", "Pothole", "Patch-Crack", "Patch-Net", "Patch-Pothole",  
"other", "Other"]
```

下载内容

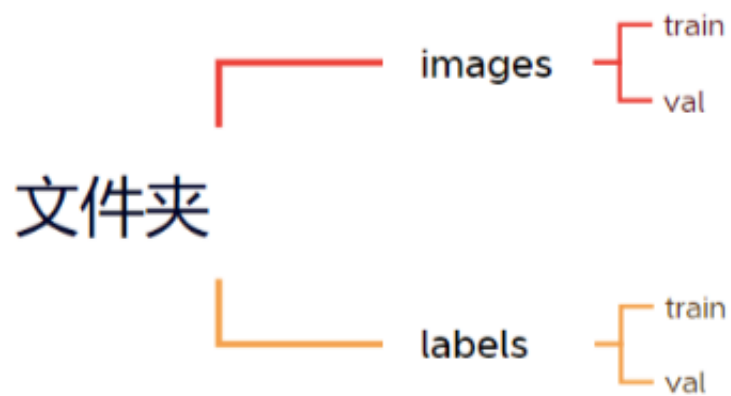
文件大小：2.47G

下载方式：百度网盘 链接: https://pan.baidu.com/s/1T9mc8zzaqSooaRzq_BXecg?pwd=96xm 提取码: 96xm

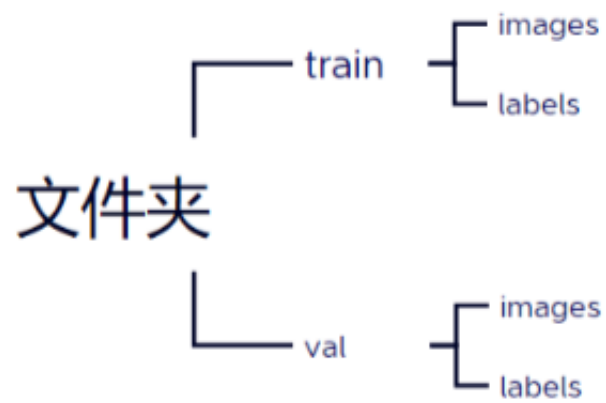
2.3 数据准备

数据集有以下两种方式放置，都可以进行训练，常见的数据集放置是第一种。

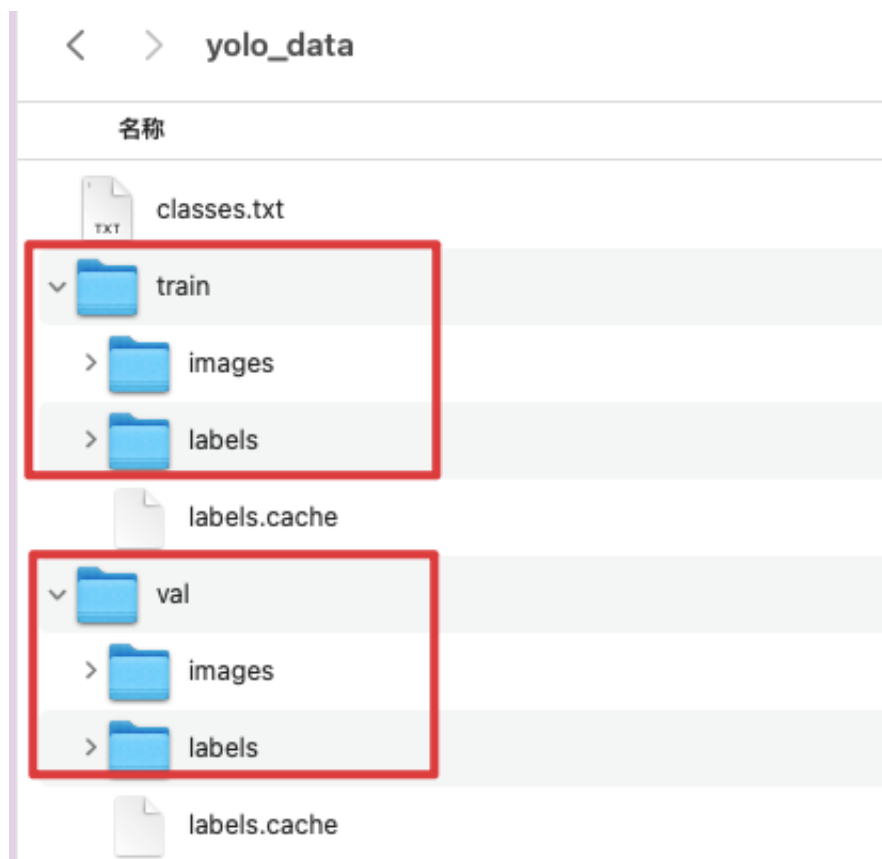
方式一：



方式二：



本数据集按照第二种方式放置：



2.4 下载代码

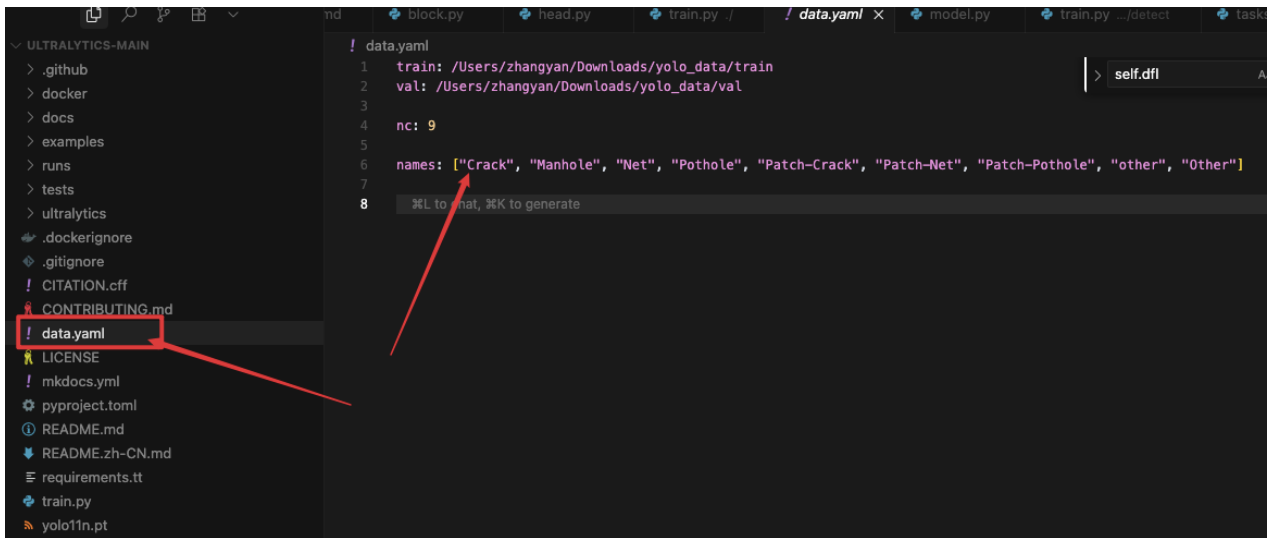
官方代码地址: <https://github.com/ultralytics/ultralytics>

本项目代码地址: <https://pan.baidu.com/s/1Pnun3TeQ45lBrjvQSPLIQg?pwd=6389>

2.5 构造data.yaml

在代码根目录下新建文件 `data.yaml`, 并编写如下内容:

```
1 train: /Users/zhangyan/Downloads/yolo_data/train #此处修改为数据实际目录
2 val: /Users/zhangyan/Downloads/yolo_data/val #此处修改为数据实际目录
3
4 nc: 9
5
6 names: ["Crack", "Manhole", "Net", "Pothole", "Patch-Crack", "Patch-Net",
  "Patch-Pothole", "other", "other"]
```



2.6 下载模型

为了方便调试运行，本项目基于yolov11 最小的版本yolov11n进行微调。

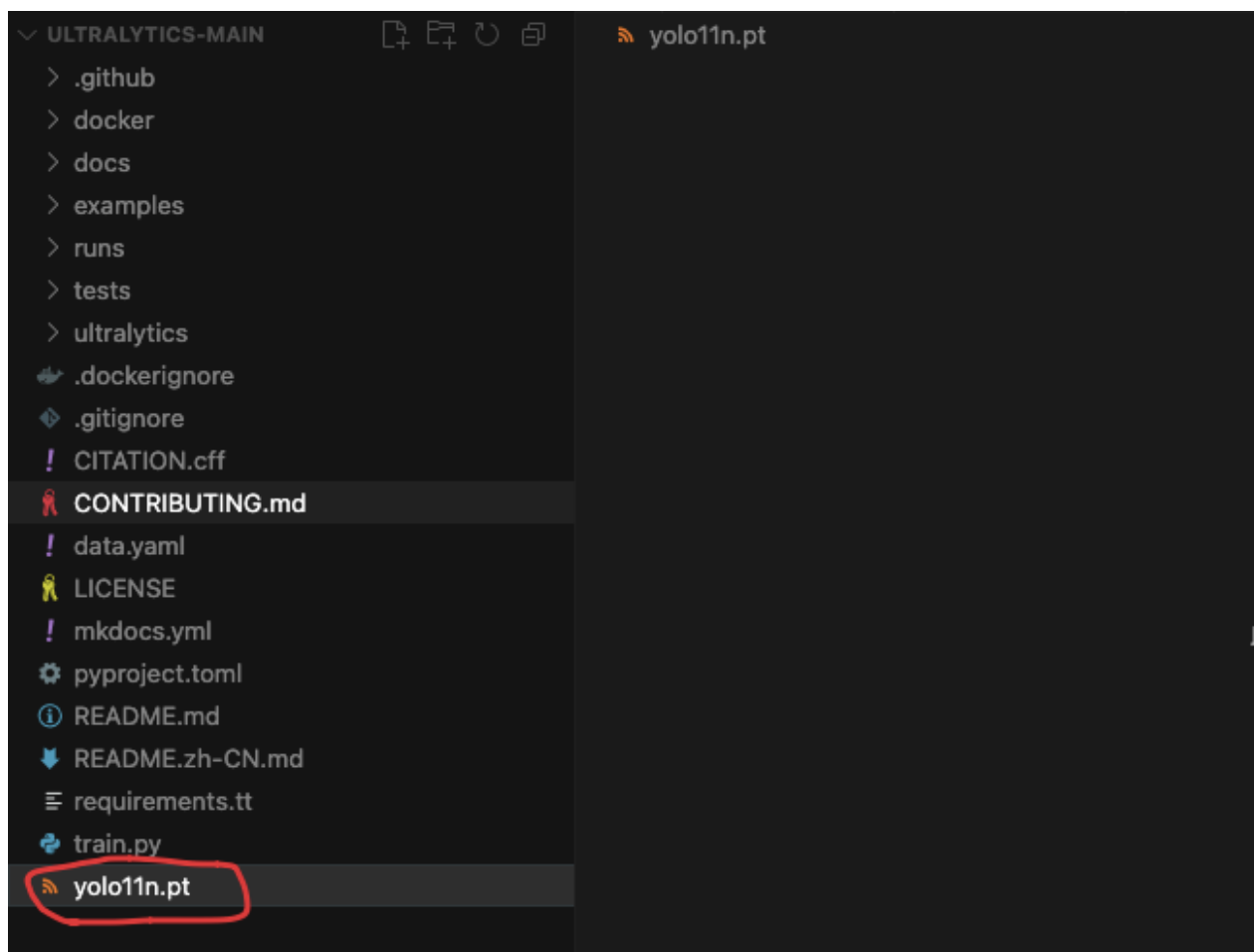
模型下载地址：<https://github.com/ultralytics/ultralytics>,

我下载目标检测

Model	size (pixels)	mAP ^{val} 50-95	Speed CPU ONNX (ms)	Speed T4 TensorRT10 (ms)	params (M)	FLOPs (B)
YOLO11n	640	39.5	56.1 ± 0.8	1.5 ± 0.0	2.6	6.5
YOLO11s	640	47.0	90.0 ± 1.2	2.5 ± 0.0	9.4	21.5
YOLO11m	640	51.5	183.2 ± 2.0	4.7 ± 0.1	20.1	68.0
YOLO11l	640	53.4	238.6 ± 1.4	6.2 ± 0.1	25.3	86.9
YOLO11x	640	54.7	462.8 ± 6.7	11.3 ± 0.2	56.9	194.9

• mAP^{val} values are for single-model single-scale on [COCO val2017](#) dataset.
Reproduce by `yolo val detect data=coco.yaml device=0`

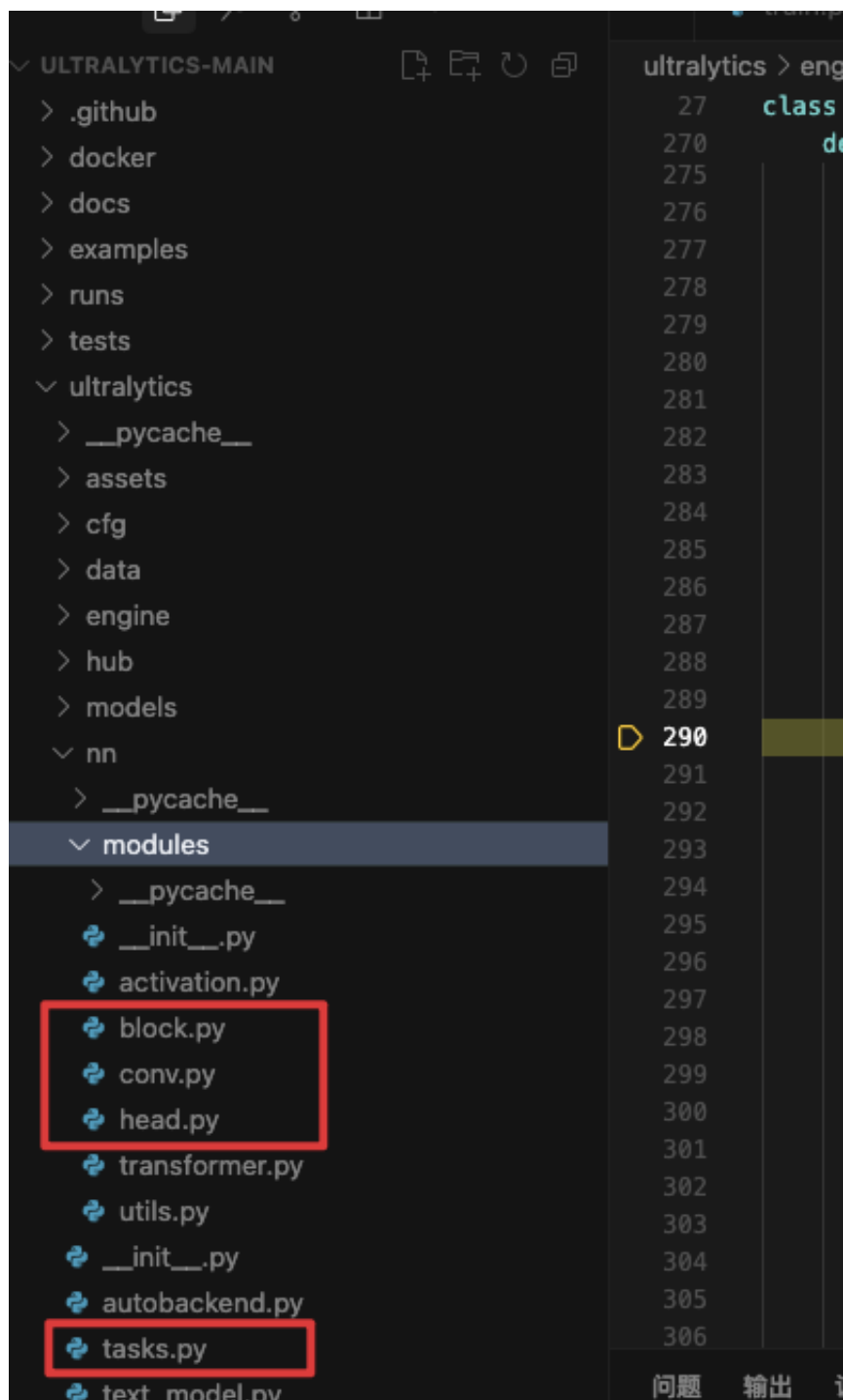
下载后将 `yolo11n.pt` 放入项目根目录：



3. 代码详解

3.1 代码结构

有三部分比较重要的代码：conv、block、tasks



conv.py

conv定义了一些基础的卷积算子结构

```

63     from .conv import (
64         CBAM,
65         ChannelAttention,
66         Concat,
67         Conv,
68         Conv2,
69         ConvTranspose,
70         DWConv,
71         DWConvTranspose2d,
72         Focus,
73         GhostConv,
74         Index,
75         LightConv,
76         RepConv,
77         SpatialAttention,
78     )

```

以最简单的卷积模块为例，他就是把 卷积层+归一化层+激活层 合在一起，然后在block中调用：

```

1  class Conv(nn.Module):
2      """
3      Standard convolution module with batch normalization and activation.
4      标准卷积模块，包含BN和激活函数。"""
5
6      default_act = nn.SiLU() # default activation
7
8      def __init__(self, c1, c2, k=1, s=1, p=None, g=1, d=1, act=True):
9          """Initialize Conv layer with given parameters."""
10         super().__init__()
11         self.conv = nn.Conv2d(c1, c2, k, s, autopad(k, p, d), groups=g,
12                                dilation=d, bias=False)
13         self.bn = nn.BatchNorm2d(c2)
14         self.act = self.default_act if act is True else act if
15         isinstance(act, nn.Module) else nn.Identity()
16
17     def forward(self, x):
18         return self.act(self.bn(self.conv(x)))
19
20     def forward_fuse(self, x):
21         """Apply convolution and activation without batch
22         normalization."""
23         return self.act(self.conv(x))

```

block.py

在block中定义了构成网络的模块形式

```

1  __all__ = (
2      "DFL",
3      "HGBlock",

```

```

4     "HGStem",
5     "SPP",
6     "SPPF",
7     "C1",
8     "C2",
9     "C3",
10    "C2f",
11    "C2fAttn",
12    "ImagePoolingAttn",
13    "ContrastiveHead",
14    "BNContrastiveHead",
15    "C3x",
16    "C3TR",
17    "C3Ghost",
18    "GhostBottleneck",
19    "Bottleneck",
20    "BottleneckCSP",
21    "Proto",
22    "RepC3",
23    "ResNetLayer",
24    "RepNCSPPELAN4",
25    "ELAN1",
26    "ADown",
27    "AConv",
28    "SPPELAN",
29    "CBFuse",
30    "CBLinear",
31    "C3k2",
32    "C2fPSA",
33    "C2PSA",
34    "RepVGGDW",
35    "CIB",
36    "C2fCIB",
37    "Attention",
38    "PSA",
39    "SCDown",
40    "TorchVision",
41 )

```

最大创新点c3k2就是在这里定义的:

```

1 class C3k2(C2f):
2     """
3     Faster Implementation of CSP Bottleneck with 2 convolutions.
4
5     可选C3k块的C2f结构。
6     """
7
8     def __init__(

```

```

9         self, c1: int, c2: int, n: int = 1, c3k: bool = False, e: float =
0.5, g: int = 1, shortcut: bool = True
10     ):
11         """
12         Initialize C3k2 module.
13
14         初始化C3k2结构。
15
16         Args:
17             c1 (int): Input channels. 输入通道数
18             c2 (int): Output channels. 输出通道数
19             n (int): Number of blocks. 块数
20             c3k (bool): Whether to use C3k blocks. 是否用C3k
21             e (float): Expansion ratio. 通道扩展比例
22             g (int): Groups for convolutions. 分组数
23             shortcut (bool): Whether to use shortcut connections. 是否使用
残差
24         """
25         super().__init__(c1, c2, n, shortcut, g, e)
26         self.m = nn.ModuleList(
27             C3k(self.c, self.c, 2, shortcut, g) if c3k else
Bottleneck(self.c, self.c, shortcut, g) for _ in range(n)
28     )

```

3.2 编写train.py

在项目根目录中新建文件 `train.py` ,编写如下代码：

```

1  from ultralytics import YOLO
2  # 加载官方预训练模型
3  model = YOLO("yolo11n.pt", task="detect") #此行打断点
4  # 模型训练
5  results = model.train(data="data.yaml", epochs=100, batch=4)

```

在实例化模型的代码行打断点，进行调试

训练参数说明

YOLOv11 模型的训练设置包括训练过程中使用的各种超参数和配置。这些设置会影响模型的性能、速度和准确性。关键的训练设置包括批量大小、学习率、动量和权重衰减。此外，优化器、损失函数和训练数据集组成的选择也会影响训练过程。对这些设置进行仔细的调整和实验对于优化性能至关重要。以下是官方给出了训练可设置参数和说明：

参数	默认值	说明
<code>model</code>	<code>None</code>	指定用于训练的模型文件。接受指向 <code>.pt</code> 预训练模型或 <code>.yaml</code> 配置文件。对于定义模型结构或初始化权重至关重要。

<code>data</code>	<code>None</code>	数据集配置文件的路径（例如 <code>coco8.yaml</code> ）。该文件包含特定于数据集的参数，包括训练数据和验证数据的路径、类名和类数。
<code>epochs</code>	<code>100</code>	训练总轮数。每个epoch代表对整个数据集进行一次完整的训练。调整该值会影响训练时间和模型性能。
<code>time</code>	<code>None</code>	最长训练时间（小时）。如果设置了该值，则会覆盖 <code>epochs</code> 参数，允许训练在指定的持续时间后自动停止。对于时间有限的训练场景非常有用。
<code>patience</code>	<code>100</code>	在验证指标没有改善的情况下，提前停止训练所需的epoch数。当性能趋于平稳时停止训练，有助于防止过度拟合。
<code>batch</code>	<code>16</code>	批量大小，有三种模式：设置为整数（例如， <code>'Batch =16 '</code> ），60% GPU内存利用率的自动模式（ <code>'Batch =-1 '</code> ），或指定利用率分数的自动模式（ <code>'Batch =0.70 '</code> ）。
<code>imgsz</code>	<code>640</code>	用于训练的目标图像尺寸。所有图像在输入模型前都会被调整到这一尺寸。影响模型精度和计算复杂度。
<code>save</code>	<code>True</code>	可保存训练检查点和最终模型权重。这对恢复训练或模型部署非常有用。
<code>save_period</code>	<code>-1</code>	保存模型检查点的频率，以 epochs 为单位。值为-1 时将禁用此功能。该功能适用于在长时间训练过程中保存临时模型。
<code>cache</code>	<code>False</code>	在内存中缓存数据集图像（ <code>True / ram</code> ）、磁盘（ <code>disk</code> ），或禁用它（ <code>False</code> ）。通过减少磁盘 I/O 提高训练速度，但代价是增加内存使用量。
<code>device</code>	<code>None</code>	指定用于训练的计算设备：单个 GPU（ <code>device=0</code> ）、多个 GPU（ <code>device=0,1</code> ）、CPU（ <code>device=cpu</code> ），或苹果芯片的 MPS（ <code>device=mps</code> ）。
<code>workers</code>	<code>8</code>	加载数据的工作线程数（每 <code>RANK</code> 多 GPU 训练）。影响数据预处理和输入模型的速度，尤其适用于多 GPU 设置。
<code>project</code>	<code>None</code>	保存训练结果的项目目录名称。允许有组织地存储不同的实验。
<code>name</code>	<code>None</code>	训练运行的名称。用于在项目文件夹内创建一个子目录，用于存储训练日志和输出结果。
<code>exist_ok</code>	<code>False</code>	如果为 <code>True</code> ，则允许覆盖现有的项目/名称目录。这对迭代实验非常有用，无需手动清除之前的输出。
<code>pretrained</code>	<code>True</code>	决定是否从预处理模型开始训练。可以是布尔值，也可以是加载权重的特定模型的字符串路径。提高训练效率和模型性能。

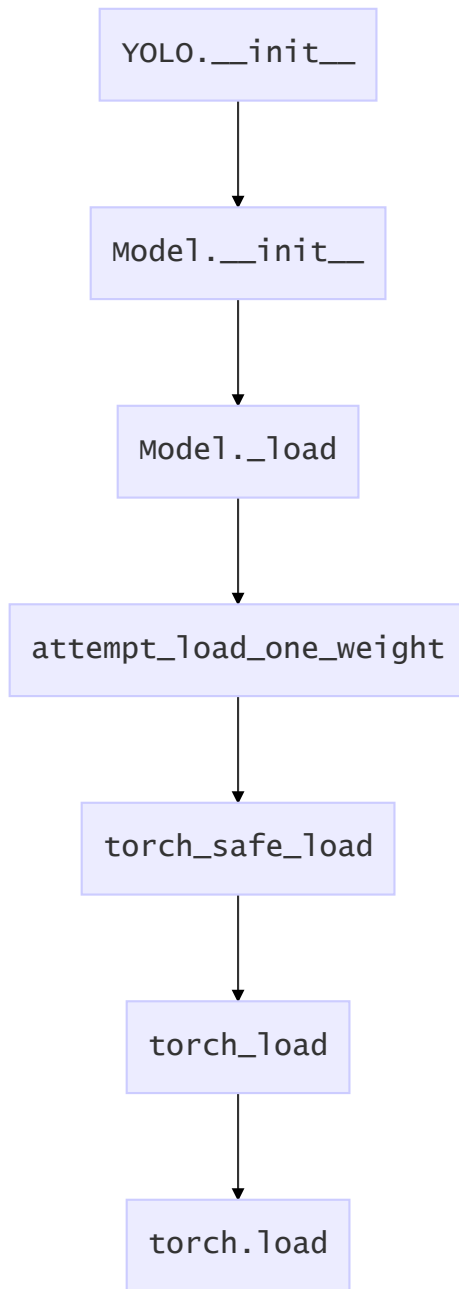
optimizer	'auto'	为训练模型选择优化器。选项包括 SGD, Adam, AdamW, NAdam, RAdam, RMSProp 等, 或 auto 用于根据模型配置进行自动选择。影响收敛速度和稳定性
verbose	False	在训练过程中启用冗长输出, 提供详细日志和进度更新。有助于调试和密切监控培训过程。
seed	0	为训练设置随机种子, 确保在相同配置下运行的结果具有可重复性。
deterministic	True	强制使用确定性算法, 确保可重复性, 但由于对非确定性算法的限制, 可能会影响性能和速度。
single_cls	False	在训练过程中将多类数据集中的所有类别视为单一类别。适用于二元分类任务, 或侧重于对象的存在而非分类。
rect	False	可进行矩形训练, 优化批次组成以减少填充。这可以提高效率和速度, 但可能会影响模型的准确性。
cos_lr	False	利用余弦学习率调度器, 根据历时的余弦曲线调整学习率。这有助于管理学习率, 实现更好的收敛。
close_mosaic	10	在训练完成前禁用最后 N 个epoch的马赛克数据增强以稳定训练。设置为 0 则禁用此功能。
resume	False	从上次保存的检查点恢复训练。自动加载模型权重、优化器状态和历时计数, 无缝继续训练。
amp	True	启用自动混合精度 (AMP) 训练, 可减少内存使用量并加快训练速度, 同时将对精度的影响降至最低。
fraction	1.0	指定用于训练的数据集的部分。允许在完整数据集的子集上进行训练, 这对实验或资源有限的情况非常有用。
profile	False	在训练过程中, 可对ONNX 和TensorRT 速度进行剖析, 有助于优化模型部署。
freeze	None	冻结模型的前 N 层或按索引指定的层, 从而减少可训练参数的数量。这对微调或迁移学习非常有用。
lr0	0.01	初始学习率 (即 SGD=1E-2, Adam=1E-3)。调整这个值对优化过程至关重要, 会影响模型权重的更新速度。
lrf	0.01	最终学习率占初始学习率的百分比 = (lr0 * lrf), 与调度程序结合使用, 随着时间的推移调整学习率。
momentum	0.937	用于 SGD 的动量因子, 或用于 Adam 优化器的 beta1, 用于将过去的梯度纳入当前更新。
weight_decay	0.0005	L2 正则化项, 对大权重进行惩罚, 以防止过度拟合。

warmup_epochs	3.0	学习率预热的历元数，学习率从低值逐渐增加到初始学习率，以在早期稳定训练。
warmup_momentum	0.8	热身阶段的初始动力，在热身期间逐渐调整到设定动力。
warmup_bias_lr	0.1	热身阶段的偏置参数学习率，有助于稳定初始历元的模型训练。
box	7.5	损失函数中边框损失部分的权重，影响对准确预测边框坐标的重视程度。
cls	0.5	分类损失在总损失函数中的权重，影响正确分类预测相对于其他部分的重要性。
dfl	1.5	分布焦点损失权重，在某些YOLO 版本中用于精细分类。
pose	12.0	姿态损失在姿态估计模型中的权重，影响着准确预测姿态关键点的关键点。
kobj	2.0	姿态估计模型中关键点对象性损失的权重，平衡检测可信度与姿态精度。
label_smoothing	0.0	应用标签平滑，将硬标签软化为目标标签和标签均匀分布的混合标签，可以提高泛化效果。
nbs	64	用于损耗正常化的标称批量大小。
overlap_mask	True	决定在训练过程中分割掩码是否应该重叠，适用于实例分割任务。
mask_ratio	4	分割掩码的下采样率，影响训练时使用的掩码分辨率。
dropout	0.0	分类任务中正则化的丢弃率，通过在训练过程中随机省略单元来防止过拟合。
val	True	可在训练过程中进行验证，以便在单独的数据集上对模型性能进行定期评估。
plots	False	生成并保存训练和验证指标图以及预测示例图，以便直观地了解模型性能和学习进度。

常用的几个训练参数是: 数据集配置文件data、训练轮数epochs、训练批次大小batch、训练使用的设备device、模型优化器optimizer、初始学习率lr0。

3.3 模型加载

调用链：




```
Torch_1_13 = True
> args = ('yolo11n.pt',)
> kwargs = {'map_location': 'cpu', 'wei...
> Globals

监视

调用堆栈 因 step 已暂停
torch_load patches.py 118:1
torch_safe_load tasks.py 1449:1
attempt_load_one_weight tasks.py
_load .../engine/model.py 295:1
__init__ .../engine/model.py 151:1
__init__ .../yolo/model.py 79:1
<module> train.py 3:1
加载多个堆栈帧

def torch_load(*args, **kwargs):
    """
    Load a PyTorch model with updated arguments to avoid warnings.

    This function wraps torch.load and adds the 'weights_only' argument for PyTorch 1.

    Args:
        *args (Any): Variable length argument list to pass to torch.load.
        **kwargs (Any): Arbitrary keyword arguments to pass to torch.load.

    Returns:
        (Any): The loaded PyTorch object.

    Notes:
        For PyTorch versions 2.0 and above, this function automatically sets 'weights_
        if the argument is not provided, to avoid deprecation warnings.
    """
    from ultralytics.utils.torch_utils import TORCH_1_13

    if TORCH_1_13 and "weights_only" not in kwargs:
        kwargs["weights_only"] = False

    return torch.load(*args, **kwargs)
```

3.4 模型构建

3.4.1 parse_model

Model.train—>Model.trainer[DetectionTrainer].get_model—>DetectionModel.init—
>parse_model

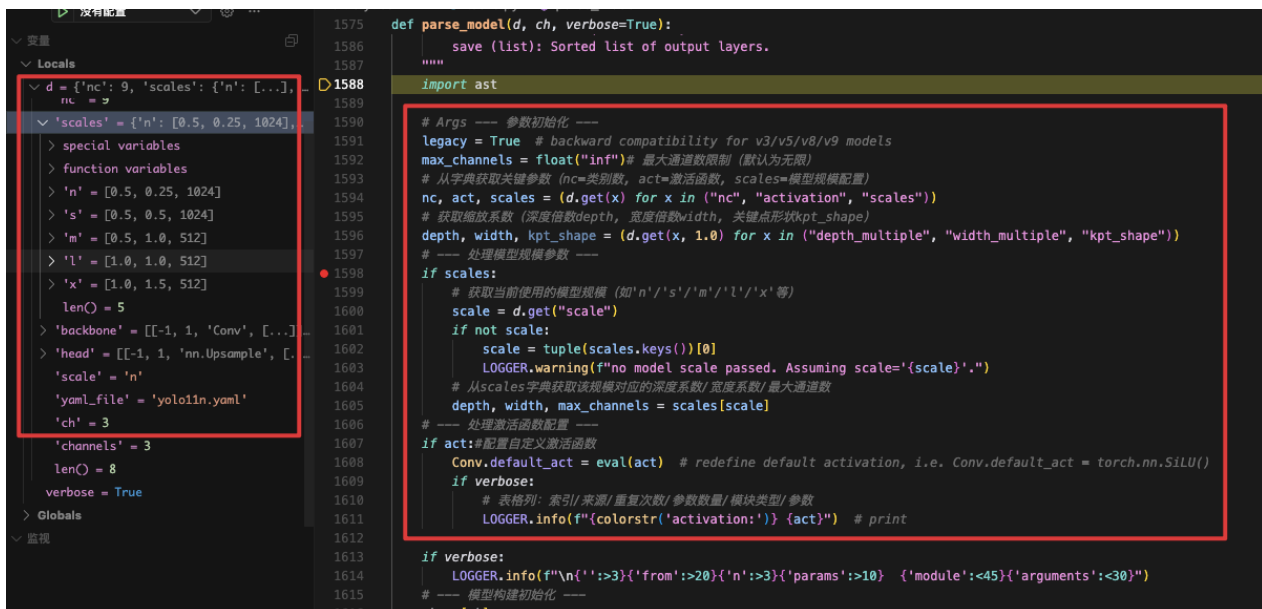
```
ultralytics > nn > tasks.py > DetectionModel > __init__
347 class DetectionModel(BaseModel):
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405

def __init__(self, cfg="yolo11n.yaml", ch=3, nc=None, verbose=True):
    """
    Initialize the YOLO detection model with the given config and parameters.

    Args:
        cfg (str | dict): Model configuration file path or dictionary.
        ch (int): Number of input channels.
        nc (int, optional): Number of classes.
        verbose (bool): Whether to display model information.
    """
    super().__init__()
    self.yaml = cfg if isinstance(cfg, dict) else yaml_model_load(cfg) # cfg dict
    if self.yaml["backbone"][0][2] == "Silence":
        LOGGER.warning(
            "YOLOv9 'Silence' module is deprecated in favor of torch.nn.Identity. "
            "Please delete local *.pt file and re-download the latest model checkpoint."
        )
        self.yaml["backbone"][0][2] = "nn.Identity"

    # Define model
    self.yaml["channels"] = ch # save channels
    if nc and nc != self.yaml["nc"]:
        LOGGER.info(f"Overriding model.yaml nc={self.yaml['nc']} with nc={nc}")
        self.yaml["nc"] = nc # override YAML value
    self.model, self.save = parse_model(deepcopy(self.yaml), ch=ch, verbose=verbose) # model
    self.names = {i: f'{i}' for i in range(self.yaml["nc"])} # default names dict
    self.inplace = self.yaml.get("inplace", True)
    self.end2end = getattr(self.model[-1], "end2end", False)

    # Build strides
```



```
def parse_model(d, ch, verbose=True):
    """
    save (list): Sorted list of output layers.
    """
    import ast

    # Args --- 参数初始化 ---
    legacy = True # backward compatibility for v3/v5/v8/v9 models
    max_channels = float("inf") # 最大通道数限制 (默认为无限)
    # 从字典获取关键参数 (nc=类别数, act=激活函数, scales=模型规模配置)
    nc, act, scales = (d.get(x) for x in ("nc", "activation", "scales"))
    # 获取缩放系数 (深度倍数depth, 宽度倍数width, 关键点形状kpt_shape)
    depth, width, kpt_shape = (d.get(x, 1.0) for x in ("depth_multiple", "width_multiple", "kpt_shape"))
    # --- 处理模型规模参数 ---
    if scales:
        # 获取当前使用的模型规模 (如'n'/'s'/'m'/'l'/'x'等)
        scale = d.get("scale")
        if not scale:
            scale = tuple(scales.keys())[0]
            LOGGER.warning(f"no model scale passed. Assuming scale='{scale}'.")
        # 从scales字典获取该规模对应的深度系数/宽度系数/最大通道数
        depth, width, max_channels = scales[scale]
    # --- 处理激活函数配置 ---
    if act: # 配置自定义激活函数
        Conv.default_act = eval(act) # redefine default activation, i.e. Conv.default_act = torch.nn.SiLU()
        if verbose:
            # 表格列: 索引/来源/重复次数/参数数量/模块类型/参数
            LOGGER.info(f"{colorstr('activation:')} {act}") # print

    if verbose:
        LOGGER.info(f"\n{'':>3}{ 'from':>20}{ 'n':>3}{ 'params':>10} {'module':<45}{ 'arguments':<30}")
    # --- 模型构建初始化 ---
    ch = [ch]
```

- nc: 9 指的是数据集集中的类别数量。
- scales: 代表模型尺寸, 分了n,s,m,l,x这5个不同大小的尺寸, 参数量依次从小到大。
-
- depth深度因子的作用: 表示模型中重复模块的数量或层数的缩放比例。这里主要用来调整C3k2模块中的子模块Bottleneck重复次数。比如主干中第一个C3k2模块的number系数是3, 我们使用0.5x3并且向上取整就等于1了, 这就代表第一个C3k2模块中Bottleneck只重复一次;
- width宽度因子的作用: 表示模型中通道数(即特征图的深度)的缩放比例,如果某个层原本有64个通道, 而width设置为0.5, 则该层的通道数变为32。比如使用yolov11n.yaml文件, 参数为[0.50, 0.25, 1024]。第一个Conv模块的输出通道数写的是64, 但是实际上这个通道数并不是64, 而是使用宽度因子 0.25x64得到的最终结果16; 同理, C3k2模块的输出通道虽然在yaml文件上写的是128, 但是在实际使用时依然要乘上宽度因子0.25, 那么第一个C3k2模块最终的到实际通道数就是0.25x128 = 32。如下图所示, 其他的依次类推。
- max-channels: 表示每层最大通道数。每层的通道数会与这个参数进行一个对比, 如果特征图通道数大于这个数, 那就取 max_channels的值。

模型构建的核心代码:

```

1677 # --- 遍历所有层配置 (骨干网络 + 检测头) ---
1678 # 每层格式: [来源, 重复次数, 模块名称, 参数列表]
1679 for i, (f, n, m, args) in enumerate(d["backbone"] + d["head"]): # from, number, module, args
1680     # 解析模块类型 (支持三种来源: torch.nn / torchvision.ops / 自定义模块)
1681     m = (
1682         getattr(torch.nn, m[3:])
1683         if "nn." in m# torch.nn中的模块 (例如nn.Conv2d)
1684         else getattr(__import__("torchvision").ops, m[16:])
1685         if "torchvision.ops." in m# torchvision中的模块
1686         else globals()[m]# 自定义模块 (从全局命名空间获取)
1687     ) # get module
1688     # 处理参数中的字符串 (替换局部变量或转换为字面值)
1689     for j, a in enumerate(args):
1690         if isinstance(a, str):
1691             with contextlib.suppress(ValueError):
1692                 # 尝试在局部变量中查找 找不到就保留原始字符串
1693                 args[j] = locals()[a] if a in locals() else ast.literal_eval(a)
1694     # 计算实际重复次数 (考虑深度缩放系数)
1695     n = n_ = max(round(n * depth), 1) if n > 1 else n # depth gain
1696     # --- 根据模块类型进行特殊处理 ---
1697     # 处理基础模块 (需要通道数计算)
1698     if m in base_modules:
1699         # c1=输入通道, c2=输出通道 (来自参数第一个元素)
1700         c1, c2 = ch[f], args[0]
1701         # 输出通道调整规则
1702         if c2 != nc: # if c2 not equal to number of classes (i.e. for Classify() output)
1703             # 1. 限制最大通道数
1704             # 2. 应用宽度倍数
1705             # 3. 按8的倍数取整 (硬件优化)
1706             c2 = make_divisible(min(c2, max_channels) * width, 8)
1707         # C2fAttn模块特殊处理 (嵌入维度和注意力头数)
1708         if m is C2fAttn: # set 1) embed channels and 2) num heads

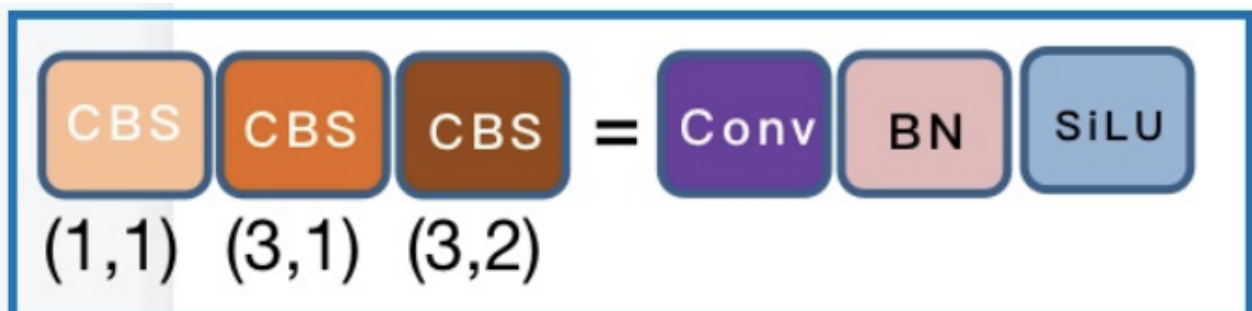
```

```

1771     c2 = ch[f] # 输出通道=上一层的输入通道
1772     # --- 模块实例化 ---
1773     # 处理模块重复构建 (n>1时创建序列结构)
1774     m_ = torch.nn.Sequential(*(m(*args) for _ in range(n)) if n > 1 else m(*args) # module
1775     # 获取模块类型名称 (简化字符串)
1776     t = str(m)[8:-2].replace("__main__.", "") # module type
1777     m_.np = sum(x.numel() for x in m_.parameters()) # number params 计算参数数量
1778     m_.i, m_.f, m_.type = i, f, t # attach index, 'from' index, type 附加元信息: 当前索引/来源索引/模块类型
1779     if verbose:
1780         LOGGER.info(f"{i:>3}{str(f):>20}{n:>3}{m_.np:10.0f} {t:<45}{str(args):<30}") # print
1781     save.extend(x % i for x in ([f] if isinstance(f, int) else f) if x != -1) # append to savelist
1782     layers.append(m_)
1783     if i == 0: # 更新通道记录 (首层初始化ch列表)
1784         ch = []
1785         ch.append(c2)
1786     # 返回Sequential模型和排序后的输出层索引
1787     return torch.nn.Sequential(*layers), sorted(save)

```

3.4.2 CBS



```

41 class Conv(nn.Module):
56     def __init__(self, c1, c2, k=1, s=1, p=None, g=1, d=1, act=True):
60         初始化标准卷积层。
61
62     Args:
63         c1 (int): Number of input channels. 输入通道数
64         c2 (int): Number of output channels. 输出通道数
65         k (int): Kernel size. 卷积核大小
66         s (int): Stride. 步长
67         p (int, optional): Padding. 填充
68         g (int): Groups. 分组数
69         d (int): Dilation. 膨胀系数
70         act (bool | nn.Module): Activation function. 激活函数
71
72     """
73     super().__init__()
74     self.conv = nn.Conv2d(c1, c2, k, s, autopad(k, p, d), groups=g, dilation=d, bias=False)
75     self.bn = nn.BatchNorm2d(c2)
76     self.act = self.default_act if act is True else act if isinstance(act, nn.Module) else nn.Identity()
77
78     def forward(self, x):
79         """
80         Apply convolution, batch normalization and activation to input tensor.
81         前向传播: 卷积->BN->激活
82
83     Args:
84         x (torch.Tensor): Input tensor. 输入张量
85
86     Returns:
87         (torch.Tensor): Output tensor. 输出张量
88     """
89     return self.act(self.bn(self.conv(x)))
90

```

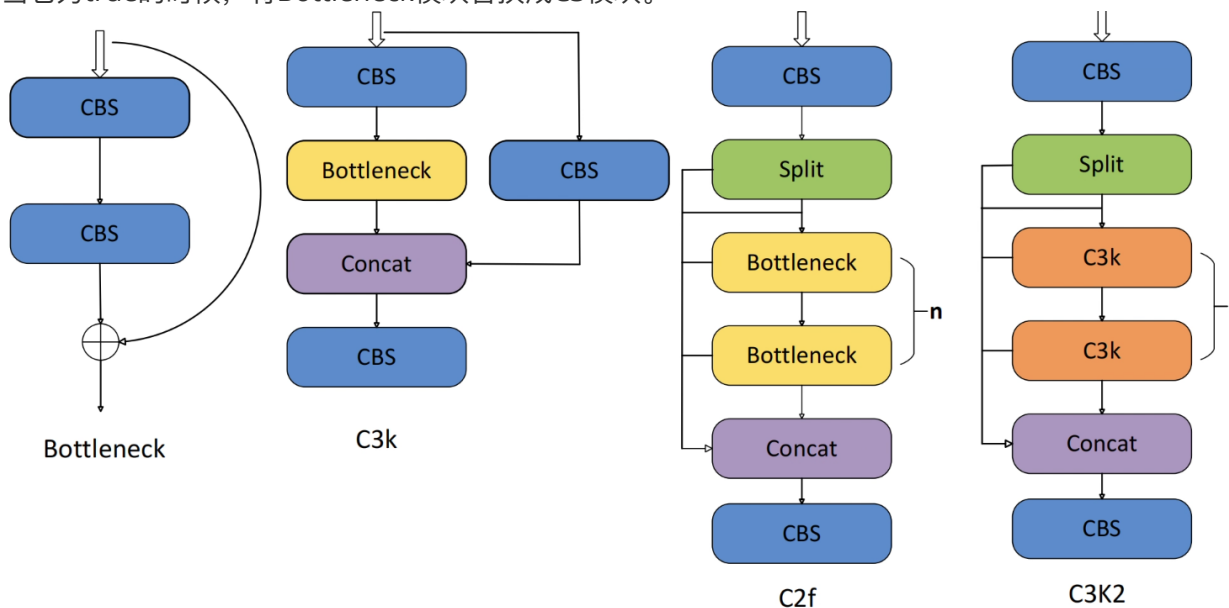
```

变量
Locals
special variables
act = True
(return) autopad = 1
c1 = 3
c2 = 16
d = 1
g = 1
k = 3
p = None
s = 2
self = Conv(
special variables
function variables
T_destination = ~T_destination
bn = BatchNorm2d(16, eps=1e-05, mome...
call_super_init = False
conv = Conv2d(3, 16, kernel_size=(3,...
default_act = SiLU()
dump_patches = False
training = True
监视

```

3.4.3 C3k2

从下面图中我们可以看到，C3K2模块其实就是C2F模块转变出来的，它代码中有一个设置，就是当c3k这个参数为FALSE的时候，C3K2模块就是C2F模块，也就是说它的Bottleneck是普通的Bottleneck；反之当它为true的时候，将Bottleneck模块替换成C3模块。



- C3k2

```
ultralitics > nn > modules > block.py > C3k2 > __init__
class C3k2(C2f):
    def __init__(
        self, c1: int, c2: int, n: int = 1, c3k: bool = False, e: float = 0.5, g: int = 1, shortcut: bool = True
    ):
        """
        Initialize C3k2 module.

        初始化C3k2结构。

        Args:
            c1 (int): Input channels. 输入通道数
            c2 (int): Output channels. 输出通道数
            n (int): Number of blocks. 块数
            c3k (bool): Whether to use C3k blocks. 是否用C3k
            e (float): Expansion ratio. 通道扩展比例
            g (int): Groups for convolutions. 分组数
            shortcut (bool): Whether to use shortcut connections. 是否使用残差
        """
        super().__init__(c1, c2, n, shortcut, g, e)
        self.m = nn.ModuleList(
            C3k(self.c, self.c, 2, shortcut, g) if c3k else Bottleneck(self.c, self.c, shortcut, g) for _ in range(n)
        )
```

• C2f

```
ultralitics > nn > modules > block.py > C2f > forward_split
class C2f(nn.Module):
    def __init__(self, c1: int, c2: int, n: int = 1, shortcut: bool = False, g: int = 1, e: float = 0.5):
        """
        """
        super().__init__()
        self.c = int(c2 * e) # hidden channels 隐藏通道数
        self.cv1 = Conv(c1, 2 * self.c, 1, 1) # 1x1降维
        self.cv2 = Conv((2 + n) * self.c, c2, 1) # 拼接后升维
        self.m = nn.ModuleList(Bottleneck(self.c, self.c, shortcut, g, k=((3, 3), (3, 3)), e=1.0) for _ in range(n))

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        """
        Forward pass through C2f layer.

        前向传播: 多分支Bottleneck 全部拼接。

        Args:
            x (torch.Tensor): 输入特征张量
        Returns:
            (torch.Tensor): 输出特征张量
        """
        y = list(self.cv1(x).chunk(2, 1)) # 通道分两支
        y.extend(m(y[-1]) for m in self.m) # 多个Bottleneck串联
        return self.cv2(torch.cat(y, 1)) # 拼接后升维
```

• Bottleneck

```
ultralitics > nn > modules > block.py > Bottleneck > __init__
class Bottleneck(nn.Module):
    def __init__(
        self,
    ):
        """
        """
        super().__init__()
        c_ = int(c2 * e) # hidden channels 隐藏通道数
        self.cv1 = Conv(c1, c_, k[0], 1)
        self.cv2 = Conv(c_, c2, k[1], 1, g=g)
        self.add = shortcut and c1 == c2

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        """
        Apply bottleneck with optional shortcut connection.

        前向传播: Bottleneck主分支和残差分支相加。

        Args:
            x (torch.Tensor): 输入特征张量
        Returns:
            (torch.Tensor): 输出特征张量
        """
        return x + self.cv2(self.cv1(x)) if self.add else self.cv2(self.cv1(x))
```

• C3k

```

ultralitics > nn > modules > block.py > C3k
1557 class C3k(C3):
1564
1575     def __init__(self, c1: int, c2: int, n: int = 1, shortcut: bool = True, g: int = 1, e: float = 0.5, k: int = 3):
1576
1577         g (int): Groups for convolutions. 分组数
1578         e (float): Expansion ratio. 通道扩展比例
1579         k (int): Kernel size. 卷积核大小
1580
1581         super().__init__(c1, c2, n, shortcut, g, e)
1582         c_ = int(c2 * e) # hidden channels 隐藏通道数
1583         # self.m = nn.Sequential(*(RepBottleneck(c_, c_, shortcut, g, k=(k, k), e=1.0) for _ in range(n)))
1584         self.m = nn.Sequential(*(Bottleneck(c_, c_, shortcut, g, k=(k, k), e=1.0) for _ in range(n)))
1585
1586 class RepVGDDW(torch.nn.Module):
1587
1588     RepVGDDW is a class that represents a depth wise separable convolutional block in RepVG architecture.
1589     深度可分离RepVGG块

```

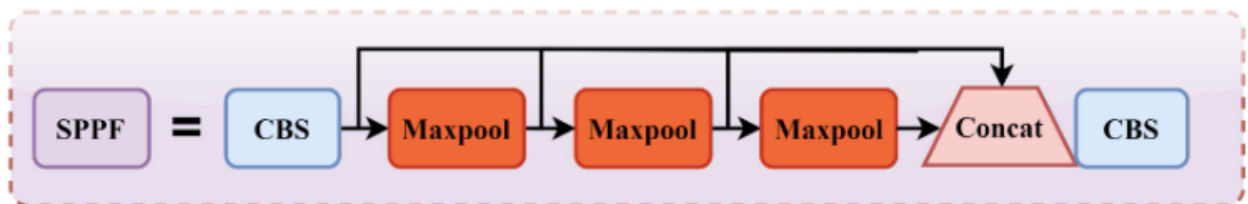
• C3

```

ultralitics > nn > modules > block.py > C3
461 class C3(nn.Module):
462
463     """
464     Initialize the CSP Bottleneck with 3 convolutions.
465     初始化C3结构。
466
467     Args:
468         c1 (int): Input channels. 输入通道数
469         c2 (int): Output channels. 输出通道数
470         n (int): Number of Bottleneck blocks. Bottleneck块数
471         shortcut (bool): Whether to use shortcut connections. 是否使用残差
472         g (int): Groups for convolutions. 分组数
473         e (float): Expansion ratio. 通道扩展比例
474
475     """
476     super().__init__()
477     c_ = int(c2 * e) # hidden channels 隐藏通道数
478     self.cv1 = Conv(c1, c_, 1, 1) # 1x1降维
479     self.cv2 = Conv(c1, c_, 1, 1) # 1x1降维
480     self.cv3 = Conv(2 * c_, c2, 1) # 拼接后升维
481     self.m = nn.Sequential(*(Bottleneck(c_, c_, shortcut, g, k=((1, 1), (3, 3)), e=1.0) for _ in range(n)))
482
483     def forward(self, x: torch.Tensor) -> torch.Tensor:
484
485         Forward pass through the CSP bottleneck with 3 convolutions.
486         前向传播: 两分支分别Bottleneck和直连, 最后拼接。
487
488         Args:
489             x (torch.Tensor): 输入特征张量
490         Returns:
491             (torch.Tensor): 输出特征张量
492
493         """
494         return self.cv3(torch.cat((self.m(self.cv1(x)), self.cv2(x)), 1)) # 拼接后升维

```

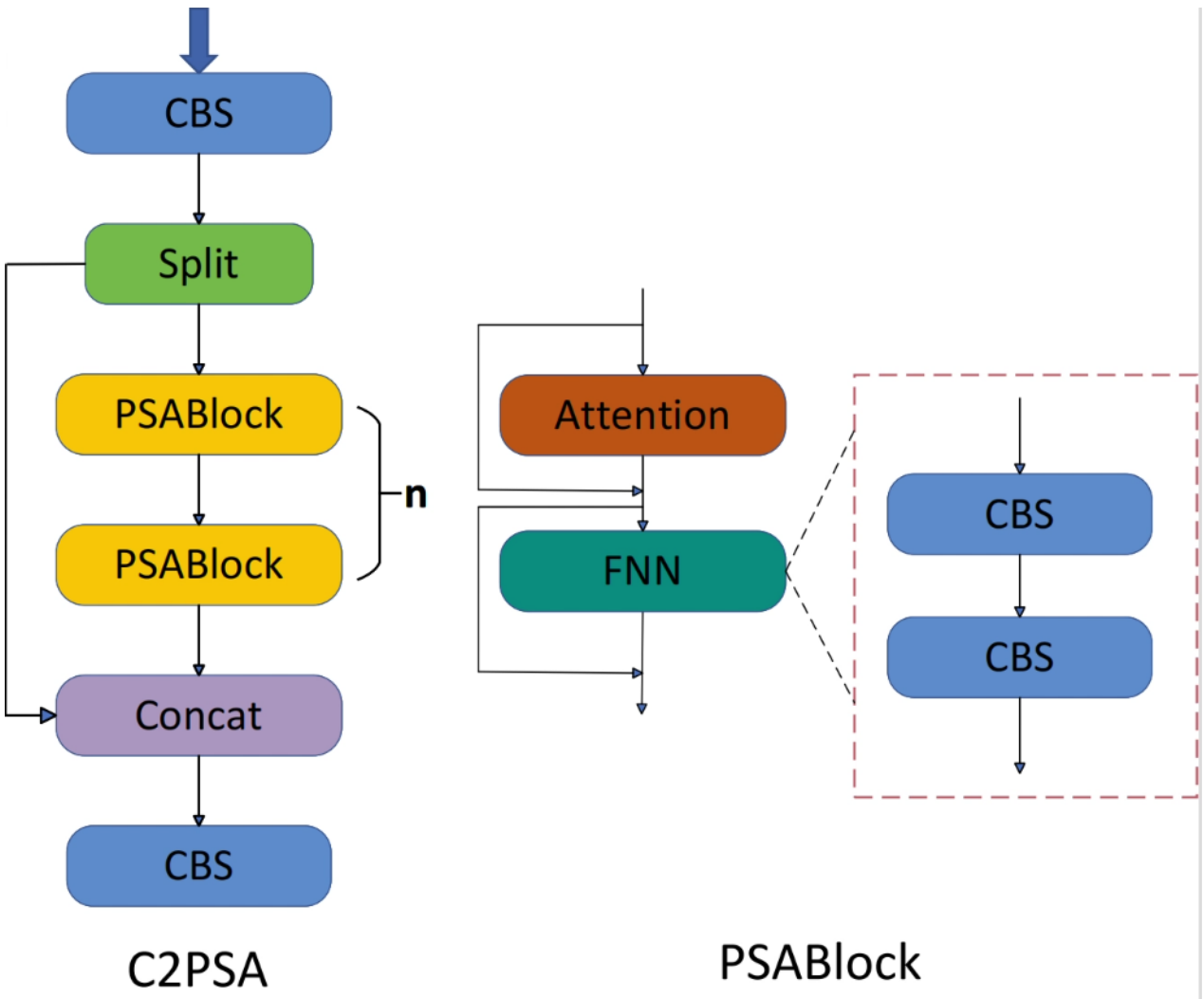
3.4.3 SPPF



```
ultralytics > nn > modules > block.py > SPPF > __init__
282 class SPPF(nn.Module):
289     def __init__(self, c1: int, c2: int, k: int = 5):
290         """
291         Initialize the SPPF layer with given input/output channels and kernel size.
292         初始化SPPF模块。
293
294         Args:
295             c1 (int): Input channels. 输入通道数
296             c2 (int): Output channels. 输出通道数
297             k (int): Kernel size. 池化核大小
298         """
299         super().__init__()
300         c_ = c1 // 2 # hidden channels 隐藏通道数
301         self.cv1 = Conv(c1, c_, 1, 1) # 1x1降维
302         self.cv2 = Conv(c_ * 4, c2, 1, 1) # 拼接后升维
303         self.m = nn.MaxPool2d(kernel_size=k, stride=1, padding=k // 2) # 固定核池化
304
305     def forward(self, x: torch.Tensor) -> torch.Tensor:
306         """
307         Apply sequential pooling operations to input and return concatenated feature maps.
308         前向传播：三次池化后拼接。
309
310         Args:
311             x (torch.Tensor): 输入特征张量
312         Returns:
313             (torch.Tensor): 输出特征张量
314         """
315         y = [self.cv1(x)]
316         y.extend(self.m(y[-1]) for _ in range(3)) # 连续三次池化
317         return self.cv2(torch.cat(y, 1))
```

3.4.4 C2PSA

C2PSA是对 `C2f` 模块的扩展，它结合了PSA(Pointwise Spatial Attention)块，用于增强特征提取和注意力机制。通过在标准 `C2f` 模块中引入 PSA 块，C2PSA实现了更强大的注意力机制，从而提高了模型对重要特征的捕捉能力。



- C2PSA

```

ultralitics > nn > modules > block.py > C2PSA > __init__
1881 class C2PSA(nn.Module):
1882     def __init__(self, c1: int, c2: int, n: int = 1, e: float = 0.5):
1883         """
1884         初始化C2PSA结构。
1885
1886         Args:
1887             c1 (int): Input channels. 输入通道数
1888             c2 (int): Output channels. 输出通道数
1889             n (int): Number of PSABlock modules. PSABlock块数
1890             e (float): Expansion ratio. 通道扩展比例
1891         """
1892         super().__init__()
1893         assert c1 == c2
1894         self.c = int(c1 * e)
1895         self.cv1 = Conv(c1, 2 * self.c, 1, 1)
1896         self.cv2 = Conv(2 * self.c, c1, 1)
1897
1898         self.m = nn.Sequential(*(PSABlock(self.c, attn_ratio=0.5, num_heads=self.c // 64) for _ in range(n)))
1899
1900     def forward(self, x: torch.Tensor) -> torch.Tensor:
1901         """
1902         Process the input tensor through a series of PSA blocks.
1903
1904         前向传播: 多PSABlock堆叠。
1905
1906         Args:
1907             x (torch.Tensor): 输入特征张量
1908         Returns:
1909             (torch.Tensor): 输出特征张量
1910         """
1911         a, b = self.cv1(x).split((self.c, self.c), dim=1)
1912         b = self.m(b)
1913         return self.cv2(torch.cat((a, b), 1))
1914
1915
1916
1917
1918
1919
1920
1921
1922
1923

```

- PSABlock

```

ultralitics > nn > modules > block.py > PSA
1796 class PSABlock(nn.Module):
1803     def __init__(self, c: int, attn_ratio: float = 0.5, num_heads: int = 4, shortcut: bool = True) -> None:
1804         """
1805         shortcut (bool): Whether to use shortcut connections. 是否使用残差
1806         """
1807         super().__init__()
1808
1809         self.attn = Attention(c, attn_ratio=attn_ratio, num_heads=num_heads)
1810         self.ffn = nn.Sequential(Conv(c, c * 2, 1), Conv(c * 2, c, 1, act=False))
1811         self.add = shortcut
1812
1813     def forward(self, x: torch.Tensor) -> torch.Tensor:
1814         """
1815         Execute a forward pass through PSABlock.
1816
1817         前向传播: 自注意力+前馈网络。
1818
1819         Args:
1820             x (torch.Tensor): 输入特征张量
1821         Returns:
1822             (torch.Tensor): 输出特征张量
1823         """
1824         x = x + self.attn(x) if self.add else self.attn(x)
1825         x = x + self.ffn(x) if self.add else self.ffn(x)
1826         return x
1827
1828
1829
1830
1831
1832
1833
1834
1835
1836

```

- Attention

```

ultralitics > nn > modules > block.py > Attention > __init__
1734 class Attention(nn.Module):
1741     def __init__(self, dim: int, num_heads: int = 8, attn_ratio: float = 0.5):
1742         """
1743         attn_ratio (float): Attention ratio for key dimension. 键向量维度缩放比, 默认为0.5
1744         """
1745         super().__init__()
1746         self.num_heads = num_heads
1747         self.head_dim = dim // num_heads
1748         self.key_dim = int(self.head_dim * attn_ratio)
1749         self.scale = self.key_dim**-0.5
1750         nh_kd = self.key_dim * num_heads
1751         h = dim + nh_kd * 2 # 计算QKV总维度: key_dim*num_heads*2(K+V) + head_dim*num_heads(Q)(dim(输入))
1752         self.qkv = Conv(dim, h, 1, act=False) # 1x1卷积生成QKV矩阵 (替代全连接层, 保留空间结构信息)
1753         self.proj = Conv(dim, dim, 1, act=False) # 输出投影层 (1x1卷积)
1754         self.pe = Conv(dim, dim, 3, 1, g=dim, act=False) # 使用分组卷积(g=dim)实现逐通道空间编码, 增强位置感知
1755
1756
1757
1758
1759
1760
1761
1762

```



```

ultralitics > nn > modules > block.py > Attention > __init__
1734 class Attention(nn.Module):
1763     def forward(self, x: torch.Tensor) -> torch.Tensor:
1764         """
1765         Forward pass of the Attention module.
1766
1767         前向传播：多头自注意力。
1768
1769         Args:
1770             x (torch.Tensor): 输入特征张量 [B, C, H, W]
1771         Returns:
1772             (torch.Tensor): 输出特征张量 [B, C, H, W]
1773         """
1774         B, C, H, W = x.shape
1775         N = H * W # 空间位置总数 (展平后)
1776         qkv = self.qkv(x) # 生成QKV [B, (C + 2*key_dim*heads), H, W]
1777         q, k, v = qkv.view(B, self.num_heads, self.key_dim * 2 + self.head_dim, N).split(#重塑并分割为Q、K、V)
1778             [self.key_dim, self.key_dim, self.head_dim], dim=2
1779         )# 结果维度: [B, num_heads, (key_dim/key_dim/head_dim), N]
1780
1781         # 计算注意力矩阵
1782         # q: [B, heads, key_dim, N] -> 转置: [B, heads, N, key_dim]
1783         # k: [B, heads, key_dim, N]
1784         # attn: [B, heads, N, N]
1785         attn = (q.transpose(-2, -1) @ k) * self.scale # QK^T / sqrt(d_k)
1786         attn = attn.softmax(dim=-1) # Softmax归一化得到注意力权重 # [B, heads, N, N]
1787         # 注意力加权求和 [8](@ref)
1788         # v: [B, heads, head_dim, N]
1789         # 结果: [B, heads, head_dim, N] -> 重塑: [B, C, H, W]
1790         # 添加位置编码 (增强空间感知)
1791         x = (v @ attn.transpose(-2, -1)).view(B, C, H, W) + self.pe(v.reshape(B, C, H, W))
1792         x = self.proj(x) # 投影层 (特征融合与降维)
1793         return x
1794

```

3.4.5 Neck

parse_model函数执行过程中会输出模型结构信息：

	from	n	params	module	arguments
0	-1	1	464	ultralitics.nn.modules.conv.Conv	[3, 16, 3, 2]
1	-1	1	4672	ultralitics.nn.modules.conv.Conv	[16, 32, 3, 2]
2	-1	1	6640	ultralitics.nn.modules.block.C3k2	[32, 64, 1, False, 0.25]
3	-1	1	36992	ultralitics.nn.modules.conv.Conv	[64, 64, 3, 2]
4	-1	1	26080	ultralitics.nn.modules.block.C3k2	[64, 128, 1, False, 0.25]
5	-1	1	147712	ultralitics.nn.modules.conv.Conv	[128, 128, 3, 2]
6	-1	1	87040	ultralitics.nn.modules.block.C3k2	[128, 128, 1, True]
7	-1	1	295424	ultralitics.nn.modules.conv.Conv	[128, 256, 3, 2]
8	-1	1	346112	ultralitics.nn.modules.block.C3k2	[256, 256, 1, True]
9	-1	1	164608	ultralitics.nn.modules.block.SPPF	[256, 256, 5]
10	-1	1	249728	ultralitics.nn.modules.block.C2PSA	[256, 256, 1]
11	-1	1	0	torch.nn.modules.upsampling.Upsample	[None, 2, 'nearest']
12	[-1, 6]	1	0	ultralitics.nn.modules.conv.Concat	[1]
13	-1	1	111296	ultralitics.nn.modules.block.C3k2	[384, 128, 1, False]
14	-1	1	0	torch.nn.modules.upsampling.Upsample	[None, 2, 'nearest']
15	[-1, 4]	1	0	ultralitics.nn.modules.conv.Concat	[1]
16	-1	1	32096	ultralitics.nn.modules.block.C3k2	[256, 64, 1, False]
17	-1	1	36992	ultralitics.nn.modules.conv.Conv	[64, 64, 3, 2]
18	[-1, 13]	1	0	ultralitics.nn.modules.conv.Concat	[1]
19	-1	1	86720	ultralitics.nn.modules.block.C3k2	[192, 128, 1, False]
20	-1	1	147712	ultralitics.nn.modules.conv.Conv	[128, 128, 3, 2]
21	[-1, 10]	1	0	ultralitics.nn.modules.conv.Concat	[1]
22	-1	1	378880	ultralitics.nn.modules.block.C3k2	[384, 256, 1, True]

- from：这个参数代表从哪一层获得输入，-1就表示从上一层获得输入，[-1, 6]就表示从上一层和第6层这两层获得输入。第一层比较特殊，这里第一层上一层 没有输入，from默认-1就好了。
- number：这个参数表示模块重复的次数，如果为2则表示该模块重复3次，这里并不一定是这个模块的重复次数，也有可能是这个模块中的子模块重复的次数。对于C3k2模块来说，这个number就代表C3k2中Bottleneck或是C3k模块重复的次数。
- module：这个就代表你这层使用的模块的名称，比如你第一层使用了Conv模块，第三层使用了C3k2模块。
- args：表示这个模块需要传入的参数，第一个参数均表示该层的输出通道数。对于第一层conv参数【64,3, 2】：64代表输出通道数，3代表卷积核大小k，2代表stride步长。每层输入通道数，默认是上一层的输出通道数。

3.4.6 Head



YOLO11在head部分的cls分支上使用深度可分离卷积，具体代码如下，cv2边界框回归分支，cv3分类分支。

```
ultralitics > nn > modules > head.py > Detect > __init__
26 class Detect(nn.Module):
83     def __init__(self, nc: int = 80, ch: Tuple = ()): # ch: backbone输出通道元组
98         self.stride = torch.zeros(self.nl) # strides computed during build
99         c2, c3 = max((16, ch[0] // 4, self.reg_max * 4), max(ch[0], min(self.nc, 100))) # channels
100         self.cv2 = nn.ModuleList(
101             nn.Sequential(Conv(x, c2, 3), Conv(c2, c2, 3), nn.Conv2d(c2, 4 * self.reg_max, 1)) for x in ch
102         ) # 回归头
103         self.cv3 = (
104             nn.ModuleList(nn.Sequential(Conv(x, c3, 3), Conv(c3, c3, 3), nn.Conv2d(c3, self.nc, 1)) for x in ch)
105         ) if self.legacy else nn.ModuleList(
106             nn.Sequential(
107                 nn.Sequential(DWConv(x, x, 3), Conv(x, c3, 1)),
108                 nn.Sequential(DWConv(c3, c3, 3), Conv(c3, c3, 1)),
109                 nn.Conv2d(c3, self.nc, 1),
110             ) for x in ch
111         )
112         ) # 类别头
113         self.dfl = DFL(self.reg_max) if self.reg_max > 1 else nn.Identity()
114
115     if self.end2end:
116         self.one2one_cv2 = copy.deepcopy(self.cv2)
117         self.one2one_cv3 = copy.deepcopy(self.cv3)
```

Detect 的 **from** 有三个数：**16**，**19**，**22**，这三个就是最终网络的输出特征图，分别对应 **P3**，**P4**，**P5**。

	from	n	params	module	arguments
0	-1	1	464	ultralitics.nn.modules.conv.Conv	[3, 16, 3, 2]
1	-1	1	4672	ultralitics.nn.modules.conv.Conv	[16, 32, 3, 2]
2	-1	1	6640	ultralitics.nn.modules.block.C3k2	[32, 64, 1, False, 0.25]
3	-1	1	36992	ultralitics.nn.modules.conv.Conv	[64, 64, 3, 2]
4	-1	1	26080	ultralitics.nn.modules.block.C3k2	[64, 128, 1, False, 0.25]
5	-1	1	147712	ultralitics.nn.modules.conv.Conv	[128, 128, 3, 2]
6	-1	1	87040	ultralitics.nn.modules.block.C3k2	[128, 128, 1, True]
7	-1	1	295424	ultralitics.nn.modules.conv.Conv	[128, 256, 3, 2]
8	-1	1	346112	ultralitics.nn.modules.block.C3k2	[256, 256, 1, True]
9	-1	1	164608	ultralitics.nn.modules.block.SPPF	[256, 256, 5]
10	-1	1	249728	ultralitics.nn.modules.block.C2PSA	[256, 256, 1]
11	-1	1	0	torch.nn.modules.upsampling.Upsample	[None, 2, 'nearest']
12	[-1, 6]	1	0	ultralitics.nn.modules.conv.Concat	[1]
13	-1	1	111296	ultralitics.nn.modules.block.C3k2	[384, 128, 1, False]
14	-1	1	0	torch.nn.modules.upsampling.Upsample	[None, 2, 'nearest']
15	[-1, 4]	1	0	ultralitics.nn.modules.conv.Concat	[1]
16	-1	1	32096	ultralitics.nn.modules.block.C3k2	[256, 64, 1, False]
17	-1	1	36992	ultralitics.nn.modules.conv.Conv	[64, 64, 3, 2]
18	[-1, 13]	1	0	ultralitics.nn.modules.conv.Concat	[1]
19	-1	1	86720	ultralitics.nn.modules.block.C3k2	[192, 128, 1, False]
20	-1	1	147712	ultralitics.nn.modules.conv.Conv	[128, 128, 3, 2]
21	[-1, 10]	1	0	ultralitics.nn.modules.conv.Concat	[1]
22	-1	1	378880	ultralitics.nn.modules.block.C3k2	[384, 256, 1, True]
23	[16, 19, 22]	1	432427	ultralitics.nn.modules.head.Detect	[9, [64, 128, 256]]

3.5 前向传播和损失计算

3.5.1 加载权重

- `model.load(weights)`

调用 `DetectionModel` 的 `load` 方法

```

141
142     model = DetectionModel(cfg, nc=self.data["nc"], ch=self.data["channels"], verbose=verbose and RANK == -1)
143     if weights:
144         model.load(weights)
145     return model
146

```

最终执行的是基类 `BaseModel` 的 `load` 方法：

```

97 class BaseModel(torch.nn.Module):
282     def _apply(self, fn):
299         m.strides = fn(m.strides)
300         return self
301
302     def load(self, weights, verbose=True):
303         """
304         Load weights into the model.
305
306         Args:
307             weights (dict | torch.nn.Module): The pre-trained weights to be loaded.
308             verbose (bool, optional): Whether to log the transfer progress.
309         """
310         model = weights["model"] if isinstance(weights, dict) else weights # torchvision models are not dicts
311         csd = model.float().state_dict() # checkpoint state_dict as FP32
312         updated_csd = intersect_dicts(csd, self.state_dict()) # intersect
313         self.load_state_dict(updated_csd, strict=False) # load
314         len_updated_csd = len(updated_csd)
315         first_conv = "model.0.conv.weight" # hard-coded to yolo models for now
316         # mostly used to boost multi-channel training
317         state_dict = self.state_dict()
318         if first_conv not in updated_csd and first_conv in state_dict:
319             c1, c2, h, w = state_dict[first_conv].shape
320             cc1, cc2, ch, cw = csd[first_conv].shape
321             if ch == h and cw == w:
322                 c1, c2 = min(c1, cc1), min(c2, cc2)
323                 state_dict[first_conv][:c1, :c2] = csd[first_conv][:c1, :c2]
324                 len_updated_csd += 1
325         if verbose:
326             LOGGER.info(f"Transferred {len_updated_csd}/{len(self.model.state_dict())} items from pretrained weights")
327

```

然后调用 `torch.nn.Module` 的 `load_state_dict` 方法。

3.5.2 构建 `dataloader` 和 `optimizer`

调用链：DetectTrainer.train—>BaseTrainer.train—>BaseTrainer._do_train—>BaseTrainer._setup_train

```

59 class BaseTrainer:
250     def _setup_train(self, world_size):
294
305         # Dataloaders
306         batch_size = self.batch_size // max(world_size, 1)
307         self.train_loader = self.get_dataloader(
308             self.data["train"], batch_size=batch_size, rank=LOCAL_RANK, mode="train"
309         )
310         if RANK in {-1, 0}:
311             # Note: When training DOTA dataset, double batch size could get OOM on images with >2000 objects.
312             self.test_loader = self.get_dataloader(
313                 self.data.get("val") or self.data.get("test"),
314                 batch_size=batch_size if self.args.task == "obb" else batch_size * 2,
315                 rank=-1,
316                 mode="val",
317             )
318             self.validator = self.get_validator()
319             metric_keys = self.validator.metrics.keys + self.label_loss_items(prefix="val")
320             self.metrics = dict(zip(metric_keys, [0] * len(metric_keys)))
321             self.ema = ModelEMA(self.model)
322             if self.args.plots:
323                 self.plot_training_labels()
324
325         # Optimizer
326         self.accumulate = max(round(self.args.nbs / self.batch_size), 1) # accumulate loss before optimizing
327         weight_decay = self.args.weight_decay * self.batch_size * self.accumulate / self.args.nbs # scale weight_decay
328         iterations = math.ceil(len(self.train_loader.dataset) / max(self.batch_size, self.args.nbs)) * self.args.epochs
329         self.optimizer = self.build_optimizer(
330             model=self.model,
331             name=self.args.optimizer,
332             lr=self.args.lr0,
333             momentum=self.args.momentum,
334             decay=weight_decay,
335             iterations=iterations,
336         )
337         # Scheduler

```

ModelEMA

模型权重在最后的n步内，会在实际的最优点处抖动，所以我们取最后n步的平均，能使得模型更加鲁棒。ema是用来在test或val时才使用，用来更新参数。

$$mv_t = decay * mv_t - 1 + (1 - decay) * variable$$

```
640 class ModelEMA:
660     def __init__(self, model, decay=0.9999, tau=2000, updates=0):
669         """
670         self.ema = deepcopy(de_parallel(model)).eval() # FP32 EMA
671         self.updates = updates # number of EMA updates
672         self.decay = lambda x: decay * (1 - math.exp(-x / tau)) # decay exponential ramp (to help early epochs)
673         for p in self.ema.parameters():
674             p.requires_grad_(False)
675         self.enabled = True
676
677     def update(self, model):
678         """
679         Update EMA parameters.
680
681         Args:
682             model (nn.Module): Model to update EMA from.
683         """
684         if self.enabled:
685             self.updates += 1
686             d = self.decay(self.updates)
687
688             msd = de_parallel(model).state_dict() # model state_dict
689             for k, v in self.ema.state_dict().items():
690                 if v.dtype.is_floating_point: # true for FP16 and FP32
691                     v *= d
692                     v += (1 - d) * msd[k].detach()
693                     # assert v.dtype == msd[k].dtype == torch.float32, f'{k}: EMA {v.dtype}, model {msd[k].dtype}'
694
```

3.5.3 训练主循环

调用链：DetectionTrainer.train—>BaseTrainer.train—>BaseTrainer._do_train

```
59 class BaseTrainer:
344     def _do_train(self, world_size=1):
345         self.plot_idx.extend([base_idx, base_idx + 1, base_idx + 4])
366         epoch = self.start_epoch
367         self.optimizer.zero_grad() # zero any resumed gradients to ensure stability on train start
368         while True:
369             self.epoch = epoch
370             self.run_callbacks("on_train_epoch_start")
371             with warnings.catch_warnings():
372                 warnings.simplefilter("ignore") # suppress 'Detected lr_scheduler.step() before optimizer.step()'
373                 self.scheduler.step()
374
375             self._model_train()
376             if RANK != -1:
377                 self.train_loader.sampler.set_epoch(epoch)
378                 pbar = enumerate(self.train_loader)
379                 # Update dataloader attributes (optional)
380                 if epoch == (self.epochs - self.args.close_mosaic):
381                     self._close_dataloader_mosaic()
382                     self.train_loader.reset()
383
384             if RANK in {-1, 0}:
385                 LOGGGER.info(self.progress_string())
386                 pbar = TQDM(enumerate(self.train_loader), total=nb)
387                 self.tloss = None
388                 for i, batch in pbar:
389                     self.run_callbacks("on_train_batch_start")
390                     # Warmup
391                     ni = i + nb * epoch
392                     if ni <= nw:
393                         xi = [0, nw] # x interp
394                         self.accumulate = max(1, int(np.interp(ni, xi, [1, self.args.nbs / self.batch_size]).round()))
395                         for j, x in enumerate(self.optimizer.param_groups):
396                             # Bias lr falls from 0.1 to lr0, all other lrs rise from 0.0 to lr0
397                             x["lr"] = np.interp(
398                                 ni, xi, [self.args.warmup_bias_lr if j == 0 else 0.0, x["initial_lr"] * self.lf(epoch)]
399                             )
```

主循环中包含：**epoch**（warmup、Forward、Backward、Optimize、Log）、**Validation**、**Save model**、**Scheduler**、**Early Stopping**

```

344     def _do_train(self, world_size=1):
345         self.loss = None
346         for i, batch in pbar:
347             self.run_callbacks("on_train_batch_start")
348             # Warmup
349             ni = i + nb * epoch
350             if ni <= nw:
351                 xi = [0, nw]  # x interp
352                 self.accumulate = max(1, int(np.interp(ni, xi, [1, self.args.nbs / self.batch_size]).round()))
353                 for j, x in enumerate(self.optimizer.param_groups):
354                     # Bias lr falls from 0.1 to lr0, all other lrs rise from 0.0 to lr0
355                     x["lr"] = np.interp(
356                         ni, xi, [self.args.warmup_bias_lr if j == 0 else 0.0, x["initial_lr"] * self.lf(epoch)]
357                     )
358                     if "momentum" in x:
359                         x["momentum"] = np.interp(ni, xi, [self.args.warmup_momentum, self.args.momentum])
360             # Forward
361             with autocast(self.amp):
362                 batch = self.preprocess_batch(batch)
363                 loss, self.loss_items = self.model(batch) #完成模型前向计算和损失计算
364                 self.loss = loss.sum()
365                 if RANK != -1:
366                     self.loss *= world_size
367                 self.tloss = (
368                     (self.tloss * i + self.loss_items) / (i + 1) if self.tloss is not None else self.loss_items
369                 )
370             # Backward
371             self.scaler.scale(self.loss).backward()

```

- preprocess_batch 准备图像数据

```

21     class DetectionTrainer(BaseTrainer):
22         def preprocess_batch(self, batch: Dict) -> Dict:
23             Preprocess a batch of images by scaling and converting to float.
24
25             Args:
26                 batch (Dict): Dictionary containing batch data with 'img' tensor.
27
28             Returns:
29                 (Dict): Preprocessed batch with normalized images.
30             """
31             batch["img"] = batch["img"].to(self.device, non_blocking=True).float() / 255
32             if self.args.multi_scale:
33                 imgs = batch["img"]
34                 sz = (
35                     random.randrange(int(self.args.imgsz * 0.5), int(self.args.imgsz * 1.5 + self.stride))
36                     // self.stride
37                     * self.stride
38                 ) # size
39                 sf = sz / max(imgs.shape[2:]) # scale factor
40                 if sf != 1:
41                     ns = [
42                         math.ceil(x * sf / self.stride) * self.stride for x in imgs.shape[2:]
43                     ] # new shape (stretched to gs-multiple)
44                     imgs = nn.functional.interpolate(imgs, size=ns, mode="bilinear", align_corners=False)
45                 batch["img"] = imgs
46             return batch

```

- model(batch) 完成模型前向计算和损失计算

```

ultralitics > nn > tasks.py > BaseModel > forward
97  class BaseModel(torch.nn.Module):
122
123      def forward(self, x, *args, **kwargs):
124          """
125          Perform forward pass of the model for either training or inference.
126
127          If x is a dict, calculates and returns the loss for training. Otherwise, returns predictions.
128
129          Args:
130              x (torch.Tensor | dict): Input tensor for inference, or dict with image tensor and
131              *args (Any): Variable length argument list.
132              **kwargs (Any): Arbitrary keyword arguments.
133
134          Returns:
135              (torch.Tensor): Loss if x is a dict (training), or network predictions (inference).
136          """
137          if isinstance(x, dict): # for cases of training and validating while training.
138              return self.loss(x, *args, **kwargs)
139          return self.predict(x, *args, **kwargs)
140

```

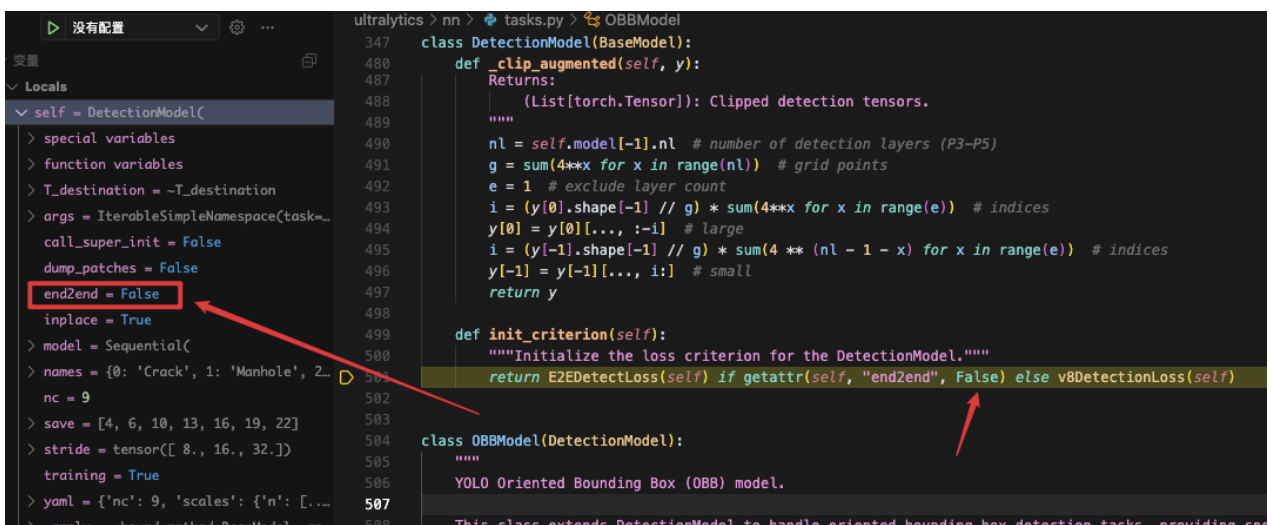
```

ultralitics > nn > tasks.py > BaseModel > init_criterion
97  class BaseModel(torch.nn.Module):
328      def loss(self, batch, preds=None):
329          """
330          Compute loss.
331
332          Args:
333              batch (dict): Batch to compute loss on.
334              preds (torch.Tensor | List[torch.Tensor], optional): Predictions.
335          """
336          if getattr(self, "criterion", None) is None:
337              self.criterion = self.init_criterion() #初始化损失函数
338
339          preds = self.forward(batch["img"]) if preds is None else preds #前向传播
340          return self.criterion(preds, batch) #损失计算
341

```

3.5.4 v8DetectionLoss实例化

执行`self.init_criterion()`, 进入 `DetectionModel` 的 `init_criterion` 方法, 其中实例化了 `v8DetectionLoss`.



```

ultralitics > nn > tasks.py > OBBModel
347  class DetectionModel(BaseModel):
480      def _clip_augmented(self, y):
481          """
482          Returns:
483              (List[torch.Tensor]): Clipped detection tensors.
484          """
485          nl = self.model[-1].nl # number of detection layers (P3-P5)
486          g = sum(4**x for x in range(nl)) # grid points
487          e = 1 # exclude layer count
488          i = (y[0].shape[-1] // g) * sum(4**x for x in range(e)) # indices
489          y[0] = y[0][..., :-1] # large
490          i = (y[-1].shape[-1] // g) * sum(4**x for x in range(e)) # indices
491          y[-1] = y[-1][..., i:] # small
492          return y
493
494      def init_criterion(self):
495          """Initialize the loss criterion for the DetectionModel."""
496          return E2EDetectLoss(self) if getattr(self, "end2end", False) else v8DetectionLoss(self)
497
498  class OBBModel(DetectionModel):
499      """
500      YOLO Oriented Bounding Box (OBB) model.
501      """
502
503      This class extends DetectionModel to handle oriented bounding box detection tasks, providing

```

进入 `v8DetectionLoss` 可以看到损失由三部分组成, 分别是: 位置预测损失: DFL + CIoU Loss, 类别预测损失: BCE Loss, 此外使用了 Task-Aligned Assigner 匹配方式。


```

class v8DetectionLoss:
    def __init__(self, model, tal_topk: int = 10): # model must be de-parallelled
        model: 去并行化后的YOLOv8模型
        tal_topk: 任务对齐分配器中选择的top-k候选框数量 默认10个

        device = next(model.parameters()).device # get model device
        h = model.args # hyperparameters 模型超参数 (包含损失权重等)

        m = model.model[-1] # Detect() module 获取最后一层 (Detect检测头模块)
        self.bce = nn.BCEWithLogitsLoss(reduction="none") # 初始化二元交叉熵损失 (带logits), 保留逐元素损失用于加权
        self.hyp = h # 存储超参数
        self.stride = m.stride # model strides 模型各层下采样步长 (如[8,16,32])
        self.nc = m.nc # number of classes 目标类别数
        self.no = m.nc + m.reg_max * 4 # 输出通道数 = 类别数 + 4*reg_max
        self.reg_max = m.reg_max # 边界框回归的高散区间数 (默认16)
        self.device = device

        self.use_dfl = m.reg_max > 1 # 是否使用分布焦点损失 (DFL), 当reg_max>1时启用
        # 任务对齐分配器 (核心改进: 结合分类置信度与IoU进行正负样本匹配)
        self.assigner = TaskAlignedAssigner(
            topk=tal_topk, # 每个真实框匹配的top-k预测框数
            num_classes=self.nc,
            alpha=0.5, # 分类权重因子
            beta=6.0 # IoU权重因子
        )
        # 边界框损失计算模块 (融合CIoU Loss和DFL)
        self.bbox_loss = BboxLoss(m.reg_max).to(device)
        # 投影矩阵: 用于DFL中高散值到连续坐标的转换 [0, 1, ..., reg_max-1]
        self.proj = torch.arange(m.reg_max, dtype=torch.float, device=device)

```

3.5.5 Forward

- 实例化v8DetectionLoss损失函数后，代码准备进行真正的Forward前向计算。

```

def loss(self, batch, preds=None):
    """
    Compute loss.

    Args:
        batch (dict): Batch to compute loss on.
        preds (torch.Tensor | List[torch.Tensor], optional): Predictions.
    """
    if getattr(self, "criterion", None) is None:
        self.criterion = self.init_criterion() # 初始化损失函数

    preds = self.forward(batch["img"]) if preds is None else preds # 前向传播
    return self.criterion(preds, batch) # 损失计算

```

- 依然调用BaseModel的forward方法，这次走到predict方法调用。

```

class BaseModel(torch.nn.Module):
    def forward(self, x, *args, **kwargs):
        """
        (torch.Tensor): Loss if x is a dict (training), or network predictions (inference).
        """
        if isinstance(x, dict): # for cases of training and validating while training.
            return self.loss(x, *args, **kwargs)
        return self.predict(x, *args, **kwargs)

```

- 由于没有打开数据增强，走到_predict_once方法。

```

class BaseModel(torch.nn.Module):
    def predict(self, x, profile=False, visualize=False, augment=False, embed=None):
        if augment: # 是否使用数据增强
            return self._predict_augment(x)
        return self._predict_once(x, profile, visualize, embed)

```

- 在_predict_once中，循环遍历model的各个子模块，进行前向计算，此时会执行各个子模块的forward方法。可以设置断点观察每一模块输出的特征图形状，对比架构图验证。

```

97 class BaseModel(torch.nn.Module):
159     def _predict_once(self, x, profile=False, visualize=False, embed=None):
172     y, dt, embeddings = [], [], [] # outputs
173     embed = frozenset(embed) if embed is not None else {-1}
174     max_idx = max(embed)
175     for m in self.model:
176         if m.f != -1: # if not from previous layer
177             x = y[m.f] if isinstance(m.f, int) else [x if j == -1 else y[j] for j in m.f] # from earlier layers
178         if profile:
179             self._profile_one_layer(m, x, dt)
180         x = m(x) # run
181         y.append(x if m.i in self.save else None) # save output
182         if visualize:
183             feature_visualization(x, m.type, m.i, save_dir=visualize)
184         if m.i in embed:
185             embeddings.append(torch.nn.functional.adaptive_avg_pool2d(x, (1, 1)).squeeze(-1).squeeze(-1)) # flatten
186             if m.i == max_idx:
187                 return torch.unbind(torch.cat(embeddings, 1), dim=0)
188     return x

```

观察输出x: [4,73,80,80]、[4,73,40,40]、[4,73,20,20]

$73=4 \times 16 + 9$

3.5.6 损失计算

Forward结束之后，进入loss计算环节：

```

ultralytics > nn > tasks.py > BaseModel > loss
97 class BaseModel(torch.nn.Module):
328     def loss(self, batch, preds=None):
333         batch (dict): Batch to compute loss on.
334         preds (torch.Tensor | List[torch.Tensor], optional): Predictions.
335         """
336         if getattr(self, "criterion", None) is None:
337             self.criterion = self.init_criterion() #初始化损失函数
338
339         preds = self.forward(batch["img"]) if preds is None else preds #前向传播
340         return self.criterion(preds, batch) #损失计算
341

```

进入 v8DetectionLoss 的 `__call__` 方法：


```

230 class v8DetectionLoss:
231     def __call__(self, preds: Any, batch: Dict[str, torch.Tensor]) -> Tuple[torch.Tensor, torch.Tensor]:
232         Calculate the sum of the loss for box, cls and dfl multiplied by batch size.
233         损失计算入口 整合边界框损失、分类损失、DFL损失
234         Args:
235             preds: 模型输出 (多尺度特征图)
236             batch: 批次数据 {images, batch_idx, cls, bboxes}
237         Returns:
238             total_loss: 加权总损失
239             loss_components: 各损失分量 [box, cls, dfl]
240         """
241         # 初始化损失分量: 边界框损失/分类损失/DFL损失
242         loss = torch.zeros(3, device=self.device) # box, cls, dfl
243         # 处理多尺度输出: 拼接特征图 [batch, no, total_anchors]
244         preds = preds[1] if isinstance(preds, tuple) else preds
245         pred_distri, pred_scores = torch.cat([xi.view(feats[0].shape[0], self.no, -1) for xi in feats], 2).split(
246             (self.reg_max * 4, self.nc), 1
247         )
248         # 调整维度: [batch, anchors, channels]
249         pred_scores = pred_scores.permute(0, 2, 1).contiguous()
250         pred_distri = pred_distri.permute(0, 2, 1).contiguous()
251
252         dtype = pred_scores.dtype
253         batch_size = pred_scores.shape[0]
254         imgsz = torch.tensor(feats[0].shape[2:], device=self.device, dtype=dtype) * self.stride[0] # image size (h,w)
255         # 生成锚点 & 计算特征图尺度
256         anchor_points, stride_tensor = make_anchors(feats, self.stride, 0.5)
257
258         # Targets 目标预处理
259         targets = torch.cat((batch["batch_idx"].view(-1, 1), batch["cls"].view(-1, 1), batch["bboxes"]), 1)
260         targets = self.preprocess(targets.to(self.device), batch_size, scale_tensor=imgsz[[1, 0, 1, 0]])
261         gt_labels, gt_bboxes = targets.split((1, 4), 2) # cls, xyxy
262         mask_gt = gt_bboxes.sum(2, keepdim=True).gt_(0.0)
263
264         # Pboxes 解码预测框

```

该方法中主要进行以下操作：

3.5.6.1 预测值切分转置

将 `preds[4, 73, xx, xx]` 切分为 `bbox[4, 64, 8400]` 距离分布分值和 `cls[4, 9, 8400]` 分值,然后转置成 `bbox[4, 8400, 64]` 和 `cls[4, 8400, 9]`

```

323     # 处理多尺度输出: 拼接特征图 [batch, no, total_anchors]
324     feats = preds[1] if isinstance(preds, tuple) else preds
325     pred_distri, pred_scores = torch.cat([xi.view(feats[0].shape[0], self.no, -1) for xi in feats], 2).split(
326         (self.reg_max * 4, self.nc), 1
327     )
328     # 调整维度: [batch, anchors, channels]
329     pred_scores = pred_scores.permute(0, 2, 1).contiguous()
330     pred_distri = pred_distri.permute(0, 2, 1).contiguous()

```

3.5.6.2 生成anchors坐标中心点

根据输入尺寸 `[640, 640]` 和下采样步长 `[8, 16, 32]` 生成每层特征图的anchors中心点坐标

```

332     dtype = pred_scores.dtype
333     batch_size = pred_scores.shape[0]
334     imgsz = torch.tensor(feats[0].shape[2:], device=self.device, dtype=dtype) * self.stride[0] # image size (h,w)
335     # 生成锚点 & 计算特征图尺度
336     anchor_points, stride_tensor = make_anchors(feats, self.stride, 0.5)

```

3.5.6.3 真是标签预处理

对输入图像的真实目标进行处理，处理成 `[640, 640]` 规格下的 `[class, x1, y1, x2, y2]`形式

```

338     # Targets 目标预处理
339     targets = torch.cat((batch["batch_idx"].view(-1, 1), batch["cls"].view(-1, 1), batch["bboxes"]), 1)
340     targets = self.preprocess(targets.to(self.device), batch_size, scale_tensor=imgsz[[1, 0, 1, 0]])
341     gt_labels, gt_bboxes = targets.split((1, 4), 2) # cls, xyxy
342     mask_gt = gt_bboxes.sum(2, keepdim=True).gt_(0.0)

```

```

269 ✓ def preprocess(self, targets: torch.Tensor, batch_size: int, scale_tensor: torch.Tensor) -> torch.Tensor:
270 ✓     """预处理真实标签：转换格式并缩放坐标到特征图尺度
271     Args:
272         targets: 原始标签 [image_idx, class, x, y, w, h]
273         batch_size: 当前批次大小
274         scale_tensor: 特征图相对于原图的缩放因子 [s_w, s_h, s_w, s_h]
275     Returns:
276         处理后的标签 [class, x1, y1, x2, y2] (xyxy格式)
277     """
278     nl, ne = targets.shape
279     if nl == 0:
280         out = torch.zeros(batch_size, 0, ne - 1, device=self.device)
281     else:
282         i = targets[:, 0] # image index 提取图像索引
283         _, counts = i.unique(return_counts=True) # 统计每张图的目标数
284         counts = counts.to(dtype=torch.int32)
285         out = torch.zeros(batch_size, counts.max(), ne - 1, device=self.device)
286         for j in range(batch_size): # 按图像索引填充数据
287             matches = i == j
288             if n := matches.sum():
289                 # 保留[class, x, y, w, h] 并转换xywh->xyxy
290                 out[j, :n] = targets[matches, 1:]
291                 out[:, 1:5] = xywh2xyxy(out[:, 1:5].mul_(scale_tensor)) # 坐标缩放：适配当前特征图尺度
292     return out

```

3.5.6.4 解码预测框

这里可以看到使用了DFL，

1. 离散概率分布建模

DFL将连续坐标值建模为离散区间上的概率分布。假设坐标真实值为 y ，将其离散化为 n 个点 $y_0, y_1, \dots, y_{n-1}$ ，模型预测每个点的概率 $P(y_i)$ 。这里的 $n=16$

2. 积分计算连续值

通过加权求和（积分）得到最终预测坐标：

$$\hat{y} = \sum_{i=0}^{n-1} P(y_i) \cdot y_i$$

```

230 class v8DetectionLoss:
231     ㄥㄣㄣㄣ
232     def bbox_decode(self, anchor_points: torch.Tensor, pred_dist: torch.Tensor) -> torch.Tensor:
233     """从预测分布解码边界框坐标 (DFL核心操作)
234     Args:
235         anchor_points: 锚点坐标 [num_anchors, 2]
236         pred_dist: 预测分布 [batch, num_anchors, 4*reg_max]
237     Returns:
238         解码后的边界框 [x1, y1, x2, y2] (xyxy格式)
239     """
240     ㄥㄣㄣㄣ
241     if self.use_dfl:
242         b, a, c = pred_dist.shape # batch, anchors, channels
243         # 重塑->softmax->积分：将离散分布转为连续坐标
244         pred_dist = pred_dist.view(b, a, 4, c // 4).softmax(3).matmul(self.proj.type(pred_dist.dtype)) #  $\sum(P(y_i) \cdot y_i)$ 
245         # pred_dist = pred_dist.view(b, a, c // 4, 4).transpose(2,3).softmax(3).matmul(self.proj.type(pred_dist.dtype))
246         # pred_dist = (pred_dist.view(b, a, c // 4, 4).softmax(2) * self.proj.type(pred_dist.dtype).view(1, 1, c, 4)).sum(3)
247         return dist2bbox(pred_dist, anchor_points, xywh=False) # 分布->边界框坐标 (相对锚点的偏移)
248     ㄥㄣㄣㄣ

```

3.5.6.5 TaskAligned Assigner 任务对齐分配

任务对齐分配：匹配预测框与真实框

代码进入 TaskAligned Assigner 类的 _forward 方法：

```

14 class TaskAlignedAssigner(nn.Module):
130     def _forward(self, pd_scores, pd_bboxes, anc_points, gt_labels, gt_bboxes, mask_gt):
149         """
150         # 步骤1: 获取正样本掩码、对齐度量和IoU
151         mask_pos, align_metric, overlaps = self.get_pos_mask(
152             pd_scores, pd_bboxes, gt_labels, gt_bboxes, anc_points, mask_gt
153         )
154         # 步骤2: 选择最高重叠的预测框 (解决多gt竞争问题)
155         target_gt_idx, fg_mask, mask_pos = self.select_highest_overlaps(mask_pos, overlaps, self.n_max_boxes)
156
157         # Assigned target
158         # 步骤3: 生成分配目标 (标签/ 边界框/ 分数)
159         target_labels, target_bboxes, target_scores = self.get_targets(gt_labels, gt_bboxes, target_gt_idx, fg_mask)
160
161         # Normalize
162         # 步骤4: 归一化对齐度量 (增强困难样本权重)
163         align_metric *= mask_pos
164         pos_align_metrics = align_metric.amax(dim=-1, keepdim=True) # b, max_num_obj # 每个gt的最大对齐度量 [b, max_gt, 1]
165         pos_overlaps = (overlaps * mask_pos).amax(dim=-1, keepdim=True) # b, max_num_obj # 每个gt的最大IoU [b, max_gt, 1]
166         # 归一化公式: align_metric * (IoU / max_align_metric)
167         norm_align_metric = (align_metric * pos_overlaps / (pos_align_metrics + self.eps)).amax(-2).unsqueeze(-1)
168         target_scores = target_scores * norm_align_metric
169
170         return target_labels, target_bboxes, target_scores, fg_mask.bool(), target_gt_idx

```

该方法中主要4个步骤:

1.get_pos_mask:获取正样本掩码、对齐度量、并为每个真实框选择top-k匹配的锚点

```

14 class TaskAlignedAssigner(nn.Module):
172     def get_pos_mask(self, pd_scores, pd_bboxes, gt_labels, gt_bboxes, anc_points, mask_gt):
193         """
194         # 步骤1: 筛选位于gt框内的锚点 [bs,max_num_obj,h*w]
195         mask_in_gts = self.select_candidates_in_gts(anc_points, gt_bboxes)
196         # Get anchor_align metric, (b, max_num_obj, h*w)
197         # 步骤2: 计算对齐度量和IoU
198         align_metric, overlaps = self.get_box_metrics(pd_scores, pd_bboxes, gt_labels, gt_bboxes, mask_in_gts * mask_gt)
199         # Get topk_metric mask, (b, max_num_obj, h*w)
200         # 步骤3: 选择每个gt的top-k锚点
201         mask_topk = self.select_topk_candidates(align_metric, topk_mask=mask_gt.expand(-1, -1, self.topk).bool())
202         # Merge all mask to a final mask, (b, max_num_obj, h*w)
203         # 综合三种条件生成最终掩码
204         mask_pos = mask_topk * mask_in_gts * mask_gt
205
206         return mask_pos, align_metric, overlaps

```

首先使用anchors的中心点坐标anc_points和真实框ltrb坐标, 计算中心点落在了哪些gt中;

```

14 class TaskAlignedAssigner(nn.Module):
347     def select_candidates_in_gts(xy_centers, gt_bboxes, eps=1e-9):
368         n_anchors = xy_centers.shape[0]
369         bs, n_boxes, _ = gt_bboxes.shape
370         # 计算锚点到gt框边界的距离 [b, n_boxes, num_anchors, 4]
371         lt, rb = gt_bboxes.view(-1, 1, 4).chunk(2, 2) # left-top, right-bottom
372         # 锚点-真实框左上角, 真实右下角-锚点
373         bbox_deltas = torch.cat((xy_centers[None] - lt, rb - xy_centers[None]), dim=2).view(bs, n_boxes, n_anchors, -1)
374         # 检查所有方向的距离是否为正 (min(dx, dy) > 0) 两个值都为正的就表明锚点在真实框内
375         return bbox_deltas.amin(3).gt_(eps)

```

然后计算对齐度量: 对齐度量公式 $align_{metric} = (class_score)^\alpha \times (IoU)^\beta$

```

14 class TaskAlignedAssigner(nn.Module):
208     def get_box_metrics(self, pd_scores, pd_bboxes, gt_labels, gt_bboxes, mask_gt):
226         overlaps (torch.Tensor): IoU overlaps between predicted and ground truth boxes.
227         """
228         na = pd_bboxes.shape[-2] # 锚点数量
229         mask_gt = mask_gt.bool() # b, max_num_obj, h*w 转换为布尔掩码
230
231         # 初始化IoU和分类分数矩阵
232         overlaps = torch.zeros([self.bs, self.n_max_boxes, na], dtype=pd_bboxes.dtype, device=pd_bboxes.device)
233         bbox_scores = torch.zeros([self.bs, self.n_max_boxes, na], dtype=pd_scores.dtype, device=pd_scores.device)
234         # 构建索引: 获取每个gt标签对应的预测分数
235         ind = torch.zeros([2, self.bs, self.n_max_boxes], dtype=torch.long) # 2, b, max_num_obj
236         ind[0] = torch.arange(end=self.bs).view(-1, 1).expand(-1, self.n_max_boxes) # b, max_num_obj
237         ind[1] = gt_labels.squeeze(-1) # b, max_num_obj
238         # Get the scores of each grid for each gt cls
239         # 提取对应类别的预测分数
240         bbox_scores[mask_gt] = pd_scores[ind[0], :, ind[1]][mask_gt] # b, max_num_obj, h*w
241
242         # (b, max_num_obj, 1, 4), (b, 1, h*w, 4) 计算IoU (仅对有效区域)
243         pd_boxes = pd_bboxes.unsqueeze(1).expand(-1, self.n_max_boxes, -1, -1)[mask_gt]
244         gt_boxes = gt_bboxes.unsqueeze(2).expand(-1, -1, na, -1)[mask_gt]
245         overlaps[mask_gt] = self.iou_calculation(gt_boxes, pd_boxes) # 计算Ciou
246         # 任务对齐度量公式: align_metric = (class_score)^alpha * (IoU)^beta
247         align_metric = bbox_scores.pow(self.alpha) * overlaps.pow(self.beta)
248         return align_metric, overlaps

```

最后, 根据对齐度量选择每个gt的top-k锚点, 这里使用了稀疏矩阵, 因为对齐度量矩阵metrics极度稀疏, 很多0值。

```

14 class TaskAlignedAssigner(nn.Module):
263     def select_topk_candidates(self, metrics, topk_mask=None):
280         """
281         # 获取top-k索引(b, max_num_obj, topk)
282         topk_metrics, topk_idxs = torch.topk(metrics, self.topk, dim=-1, largest=True)
283         # 生成top-k掩码 (防止选择无效候选)
284         if topk_mask is None:
285             topk_mask = (topk_metrics.max(-1, keepdim=True)[0] > self.eps).expand_as(topk_idxs)
286         # 构建稀疏矩阵标记top-k位置(b, max_num_obj, topk)
287         topk_idxs.masked_fill_(~topk_mask, 0)
288
289         # (b, max_num_obj, topk, h*w) -> (b, max_num_obj, h*w)
290         count_tensor = torch.zeros(metrics.shape, dtype=torch.int8, device=topk_idxs.device)
291         ones = torch.ones_like(topk_idxs[:, :, :1], dtype=torch.int8, device=topk_idxs.device)
292         for k in range(self.topk): # 通过scatter_add高效标记top-k
293             # Expand topk_idxs for each value of k and add 1 at the specified positions
294             count_tensor.scatter_add_(-1, topk_idxs[:, :, k : k + 1], ones)
295         # Filter invalid bboxes 过滤重复标记 (确保每个位置只被计数一次)
296         count_tensor.masked_fill_(count_tensor > 1, 0)
297
298         return count_tensor.to(metrics.dtype)

```

2.select_highest_overlaps:选择最高重叠的预测框 (解决多gt竞争问题)

```

14 class TaskAlignedAssigner(nn.Module):
378     def select_highest_overlaps(mask_pos, overlaps, n_max_boxes):
396         """
397         # Convert (b, n_max_boxes, h*w) -> (b, h*w)
398         fg_mask = mask_pos.sum(-2)
399         # 处理多gt匹配的锚点 (选择IoU最高的gt)
400         if fg_mask.max() > 1: # one anchor is assigned to multiple gt_bboxes
401             mask_multi_gts = (fg_mask.unsqueeze(1) > 1).expand(-1, n_max_boxes, -1) # (b, n_max_boxes, h*w)
402             max_overlaps_idx = overlaps.argmax(1) # (b, h*w)
403             # 构建最高IoU的one-hot编码
404             is_max_overlaps = torch.zeros(mask_pos.shape, dtype=mask_pos.dtype, device=mask_pos.device)
405             is_max_overlaps.scatter_(1, max_overlaps_idx.unsqueeze(1), 1)
406             # 更新掩码: 仅保留最高IoU的gt
407             mask_pos = torch.where(mask_multi_gts, is_max_overlaps, mask_pos).float() # (b, n_max_boxes, h*w)
408             fg_mask = mask_pos.sum(-2)
409         # Find each grid serve which gt(index)
410         target_gt_idx = mask_pos.argmax(-2) # (b, h*w)
411         return target_gt_idx, fg_mask, mask_pos

```

3.get_targets:生成分配目标（标签/边界框/分数））

```
14 class TaskAlignedAssigner(nn.Module):
300     def get_targets(self, gt_labels, gt_bboxes, target_gt_idx, fg_mask):
321         # 转换gt索引为全局索引Assigned target labels, (b, 1)
322         batch_ind = torch.arange(end=self.bs, dtype=torch.int64, device=gt_labels.device)[..., None]
323         target_gt_idx = target_gt_idx + batch_ind * self.n_max_boxes # (b, h*w)
324         #提取分配的标签和边界框
325         target_labels = gt_labels.long().flatten()[target_gt_idx] # (b, h*w)
326
327         # Assigned target boxes, (b, max_num_obj, 4) -> (b, h*w, 4)
328         target_bboxes = gt_bboxes.view(-1, gt_bboxes.shape[-1])[target_gt_idx]
329
330         # Assigned target scores
331         target_labels.clamp_(0)
332
333         #生成目标分数矩阵 (高效替代one_hot) 10x faster than F.one_hot()
334         target_scores = torch.zeros(
335             (target_labels.shape[0], target_labels.shape[1], self.num_classes),
336             dtype=torch.int64,
337             device=target_labels.device,
338         ) # (b, h*w, 80)
339         target_scores.scatter_(2, target_labels.unsqueeze(-1), 1) # 在类别维度填充1
340         #应用前景掩码 (背景位置置零)
341         fg_scores_mask = fg_mask[:, :, None].repeat(1, 1, self.num_classes) # (b, h*w, 80)
342         target_scores = torch.where(fg_scores_mask > 0, target_scores, 0)
343
344         return target_labels, target_bboxes, target_scores
```

4.归一化对齐度量（增强困难样本权重）

归一化公式：align_metric * IoU / max_align_metric

3.5.6.6 分类损失计算

类损失：BCE二元交叉熵（仅正样本）

```
361         # Cls loss
362         # loss[1] = self.varifocal_loss(pred_scores, target_scores, target_labels) / target_scores_sum # VFL way
363         # 1. 分类损失: 二元交叉熵 (仅正样本)
364         loss[1] = self.bce(pred_scores, target_scores.to(dtype)).sum() / target_scores_sum # BCE
```

3.5.6.7 Bbox loss计算

边界框损失：含CloU Loss + DFL（仅正样本）


```

108 class BboxLoss(nn.Module):
135     def forward(
152         """
153         # ===== IoU损失计算 =====
154         # 计算样本权重: 基于分类置信度加权 (困难样本权重更高)
155         # target_scores.sum(-1): 每个锚点的类别分数总和 [bs, num_anchors]
156         # [fg_mask]: 仅选取前景样本 [num_fg_samples]
157         weight = target_scores.sum(-1)[fg_mask].unsqueeze(-1)
158         # 计算CIoU (Complete IoU) [1,4](@ref)
159         # 仅在前景样本上计算, 减少计算量
160         iou = bbox_iou(pred_bboxes[fg_mask], target_bboxes[fg_mask], xywh=False, CIoU=True)
161         # IoU损失公式: (1 - IoU) * 权重
162         loss_iou = ((1.0 - iou) * weight).sum() / target_scores_sum
163
164         # DFL loss
165         # ===== DFL损失计算 =====
166         if self.dfl_loss:
167             # 将真实框转换为距离表示 (ltrb格式)
168             target_ltrb = bbox2dist(anchor_points, target_bboxes, self.dfl_loss.reg_max - 1)
169             # 重塑预测分布: [num_fg_samples, reg_max] 提取真实距离: [num_fg_samples, 4] 计算损失并加权
170             loss_dfl = self.dfl_loss(pred_dist[fg_mask].view(-1, self.dfl_loss.reg_max), target_ltrb[fg_mask]) * weight
171             loss_dfl = loss_dfl.sum() / target_scores_sum # 归一化
172         else:
173             loss_dfl = torch.tensor(0.0).to(pred_dist.device)
174
175         return loss_iou, loss_dfl

```

- dfl损失计算

实例设定

假设真实坐标值为 $y = 5.3$, 离散区间间隔为1, 相邻离散点为 $y_i = 5$ 和 $y_{i+1} = 6$ 。模型预测这两个点概率分别为:

$$S_i = 0.4, \quad S_{i+1} = 0.3$$

其他14个离散点的概率总和为 $1 - 0.4 - 0.3 = 0.3$, 但DFL仅优化相邻两点。

损失计算步骤

1. 计算权重系数

真实值与离散点的距离权重为:

$$S_i^{\text{target}} = \frac{y_{i+1} - y}{y_{i+1} - y_i} = \frac{6 - 5.3}{1} = 0.7$$

$$S_{i+1}^{\text{target}} = \frac{y - y_i}{y_{i+1} - y_i} = \frac{5.3 - 5}{1} = 0.3$$

2. 代入DFL公式

损失值为:

$$\text{DFL} = -(0.7 \cdot \log(0.4) + 0.3 \cdot \log(0.3))$$

3. 计算对数项

$$\log(0.4) \approx -0.9163, \quad \log(0.3) \approx -1.2039$$

4. 最终结果

$$\text{DFL} = -(0.7 \times (-0.9163) + 0.3 \times (-1.2039)) = -(-0.6414 - 0.3612) = 1.0026$$

```

87  class DFloss(nn.Module):
88      """Criterion class for computing Distribution Focal Loss (DFL)."""
89
90  def __init__(self, reg_max: int = 16) -> None:
91      """Initialize the DFL module with regularization maximum."""
92      super().__init__()
93      self.reg_max = reg_max
94
95  def __call__(self, pred_dist: torch.Tensor, target: torch.Tensor) -> torch.Tensor:
96      """Return sum of left and right DFL losses from https://ieeexplore.ieee.org/document/9792391."""
97      target = target.clamp_(0, self.reg_max - 1 - 0.01)
98      tl = target.long() # target left
99      tr = tl + 1 # target right
100     wl = tr - target # weight left
101     wr = 1 - wl # weight right
102     return (
103         F.cross_entropy(pred_dist, tl.view(-1), reduction="none").view(tl.shape) * wl
104         + F.cross_entropy(pred_dist, tr.view(-1), reduction="none").view(tl.shape) * wr
105     ).mean(-1, keepdim=True)

```